

Generative AI

draft slides, do not print

Computer Vision and Artificial
Intelligence

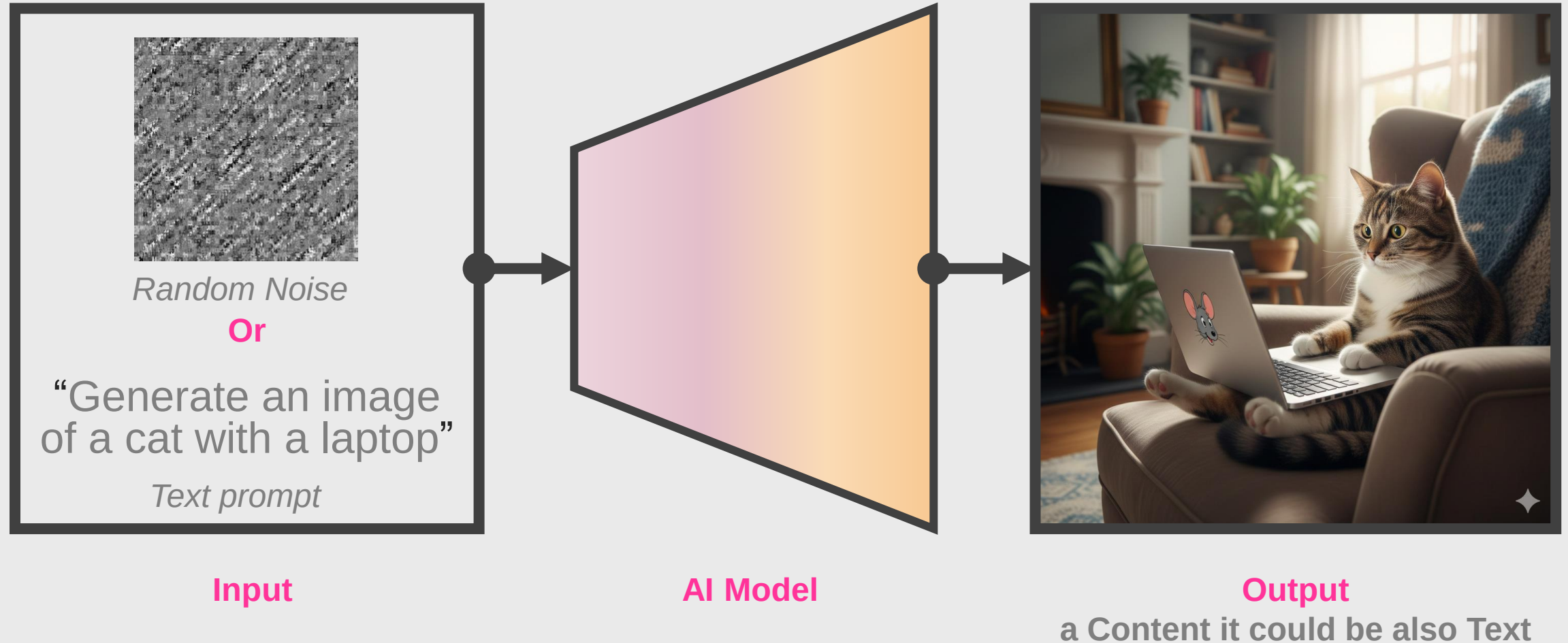
Dr Varun Ojha

varun.ojha@ncl.ac.uk

School of Computing
Newcastle University

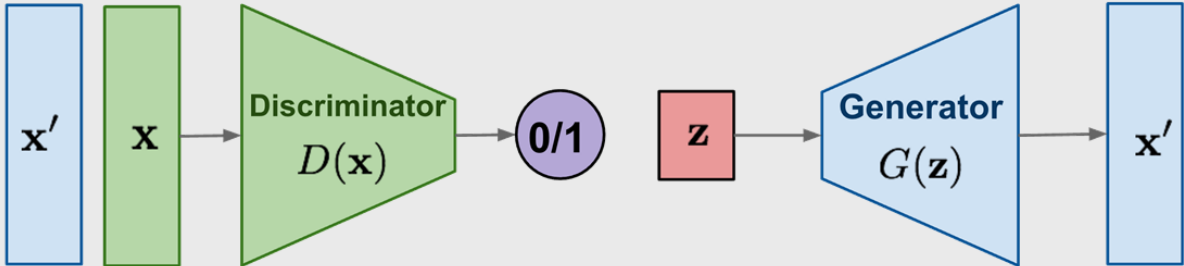


Generative AI

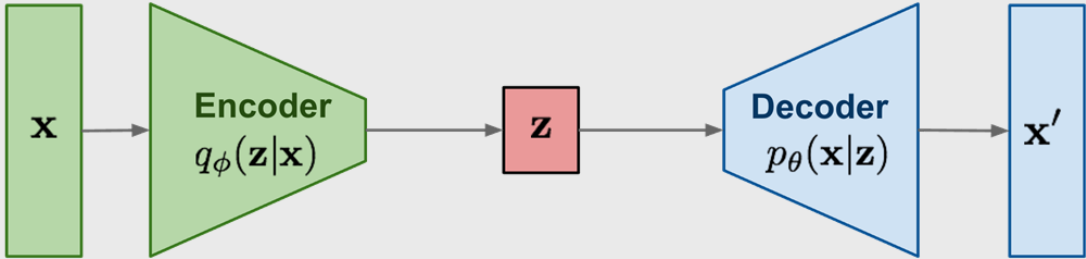


Generative AI

GAN: Adversarial training



VAE: maximize variational lower bound



Diffusion models:
Gradually add Gaussian noise and then reverse



Generative AI

3:33 PM

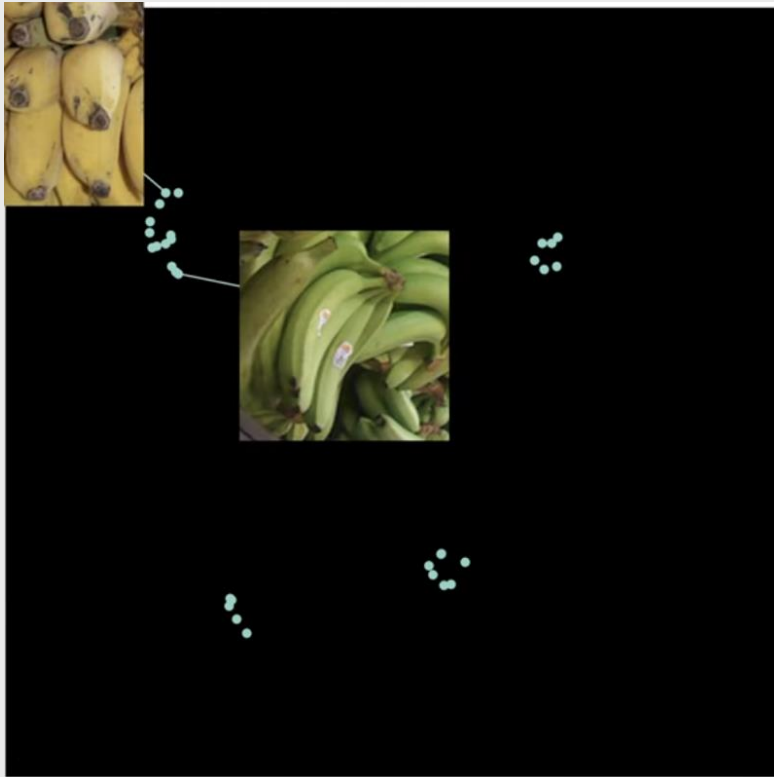
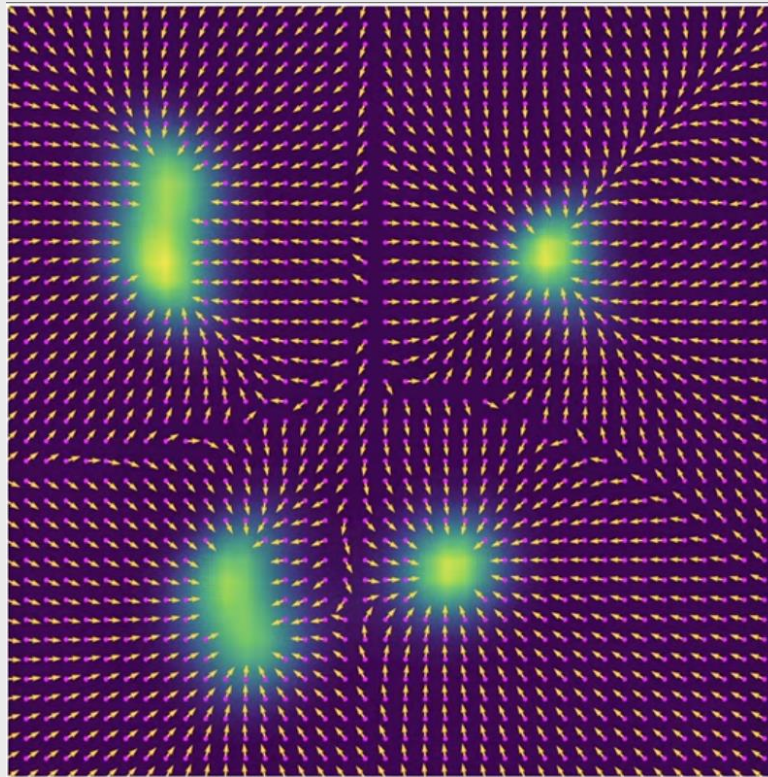
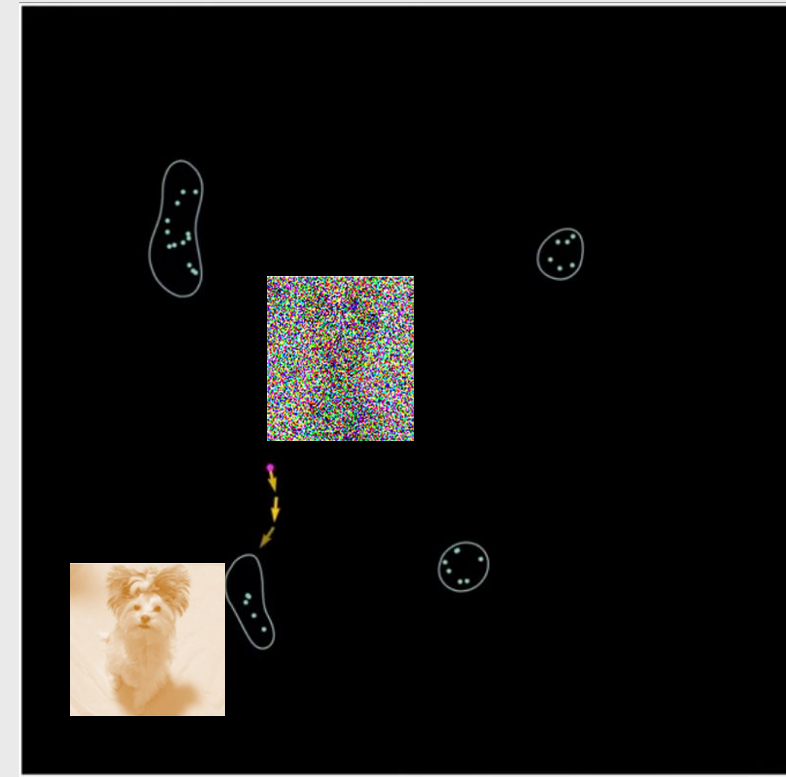


Image Input Space



Learned Gradient Space

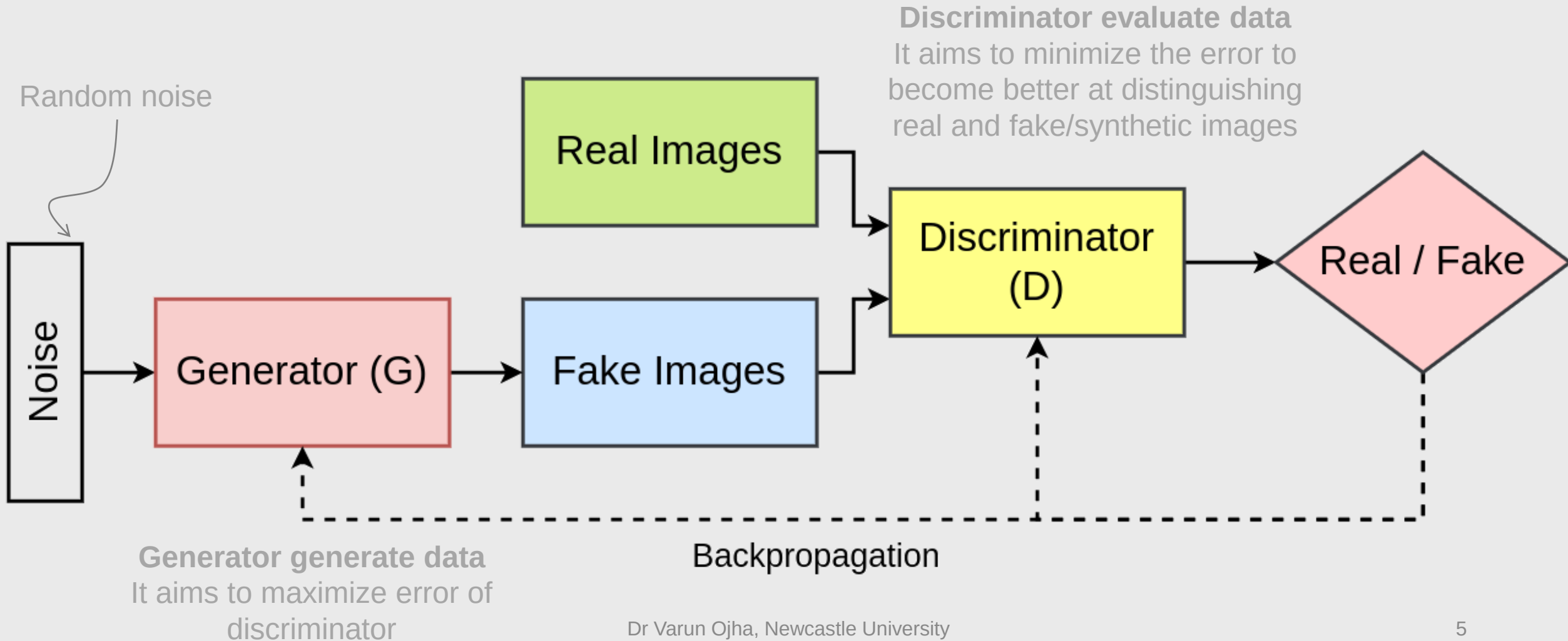


Learned Noise to Image

Generative Adversarial Networks

3:33 PM

The aim of the GAN generates realistic data from



Generative Adversarial Nets

**Ian J. Goodfellow, Jean Pouget-Abadie*, Mehdi Mirza, Bing Xu, David Warde-Farley,
Sherjil Ozair[†], Aaron Courville, Yoshua Bengio[‡]**

Département d'informatique et de recherche opérationnelle
Université de Montréal
Montréal, QC H3C 3J7

Abstract

We propose a new framework for estimating generative models via an adversarial process, in which we simultaneously train two models: a generative model G that captures the data distribution, and a discriminative model D that estimates the probability that a sample came from the training data rather than G . The training procedure for G is to maximize the probability of D making a mistake. This framework corresponds to a minimax two-player game. In the space of arbitrary functions G and D , a unique solution exists, with G recovering the training data distribution and D equal to $\frac{1}{2}$ everywhere. In the case where G and D are defined by multilayer perceptrons, the entire system can be trained with backpropagation. There is no need for any Markov chains or unrolled approximate inference networks during either training or generation of samples. Experiments demonstrate the potential of the framework through qualitative and quantitative evaluation of the generated samples.

https://papers.nips.cc/paper_files/paper/2014/hash/f033ed80deb0234979a61f95710dbe25-Abstract.html

Generative Adversarial Networks

Image: Bishop, Deep Learning



Random noise

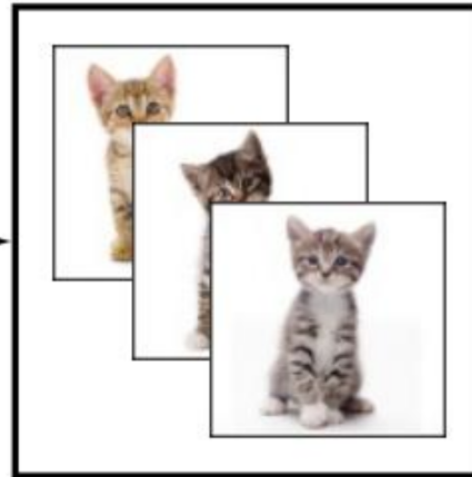
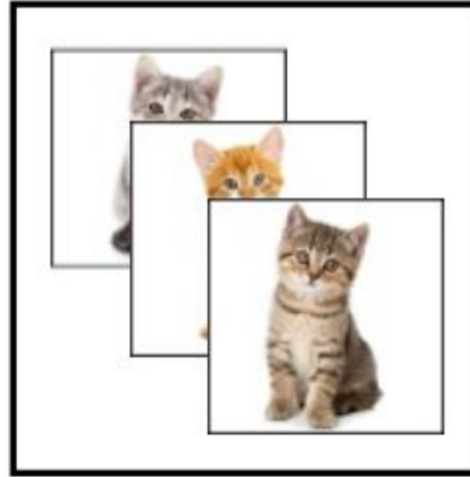
$\mathbf{z}$

Generator

$g(\mathbf{z}, \mathbf{w})$

Generator aims to maximize error of discriminator

real images



synthetic images

Discriminator

$d(\mathbf{x}, \phi)$

Discriminator, is a **binary classifier** that aims to minimize the error to become better at distinguishing real and fake/synthetic images

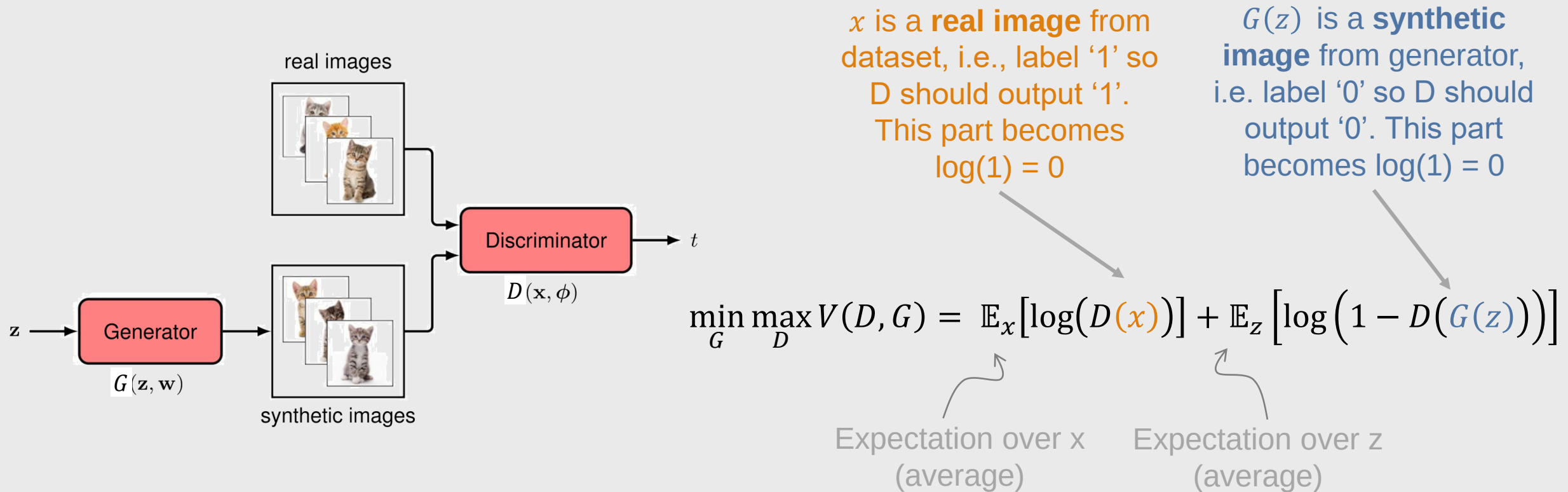
t

3:33 PM

Generative Adversarial Networks

3:33 PM

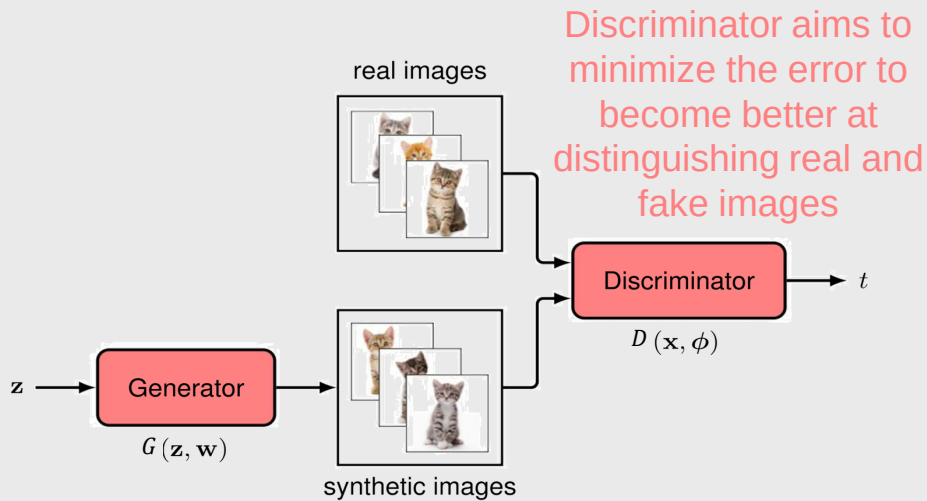
min-max loss



Generative Adversarial Networks

min-max loss

3:33 PM



D to maximize the probability of assigning the correct label to both training examples and samples from G by maximizing:

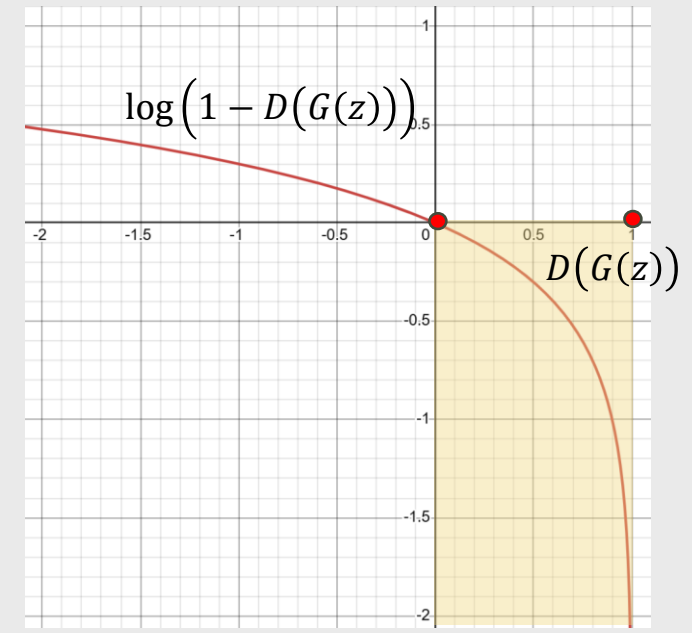
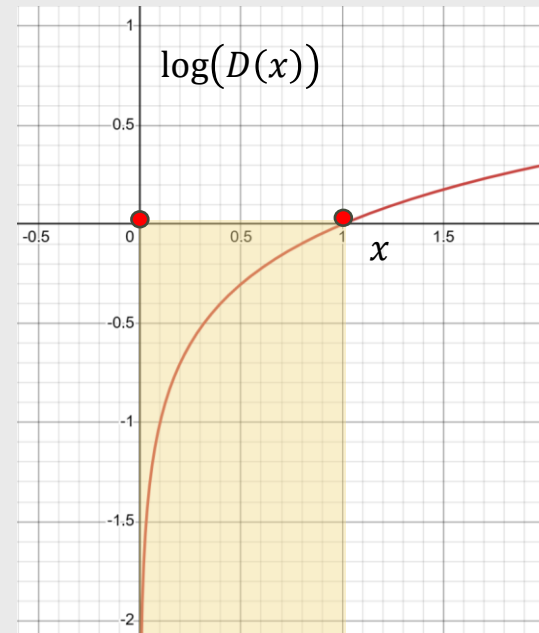
$$\mathbb{E}_x[\log(D(x))] + \mathbb{E}_z[\log(1 - D(G(z)))]$$

Low value means discriminator can identify the real vs fake i.e., if $D(x) = 1$ and $D(G(z)) = 0$, then we have $\log 1 + \log 1 = 0 \rightarrow$ **Discriminator Wins.**

D is a function of $\log(D(x))$ vs (x)

D is a function of $\log(1 - D(G(z)))$ vs $D(G(z))$

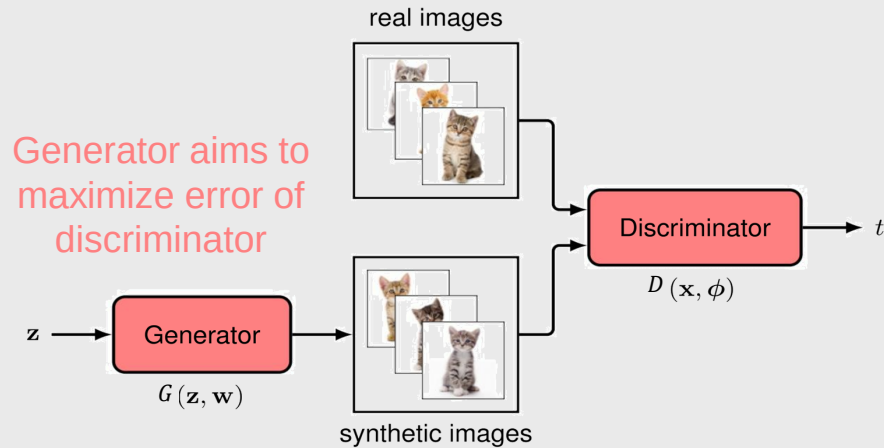
$$\max_D V(D, G) = \mathbb{E}_x[\log(D(x))] + \mathbb{E}_z[\log(1 - D(G(z)))]$$



Generative Adversarial Networks

3:33 PM

min-max loss



G to minimize:

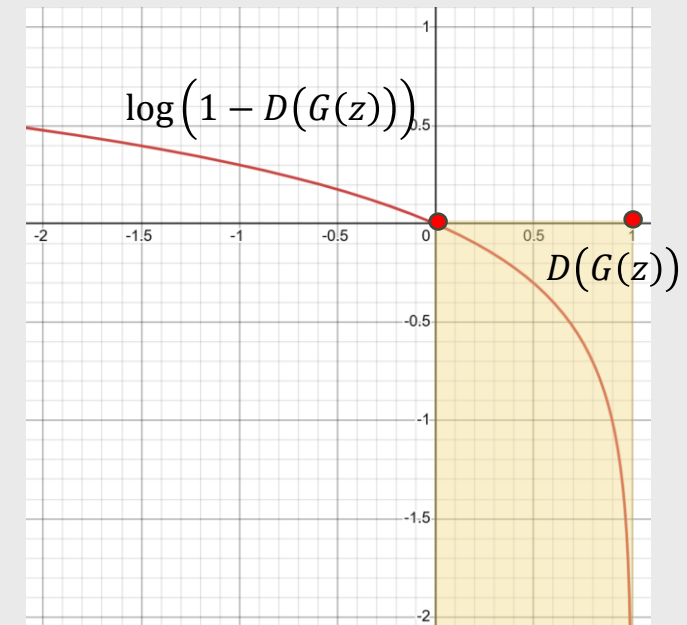
$$\mathbb{E}_z [\log(1 - D(G(z)))]$$

Low value means generator create realistic images, i.e., if $D(G(z))$ close to 1 then generator fools the Discriminator, and Generator Wins

We would $\log(1 - 0.99) = \log(0.01) = -2 \rightarrow$ **Generator Wins**.

G is a function of
 $\log(1 - D(G(z)))$ vs $D(G(z))$

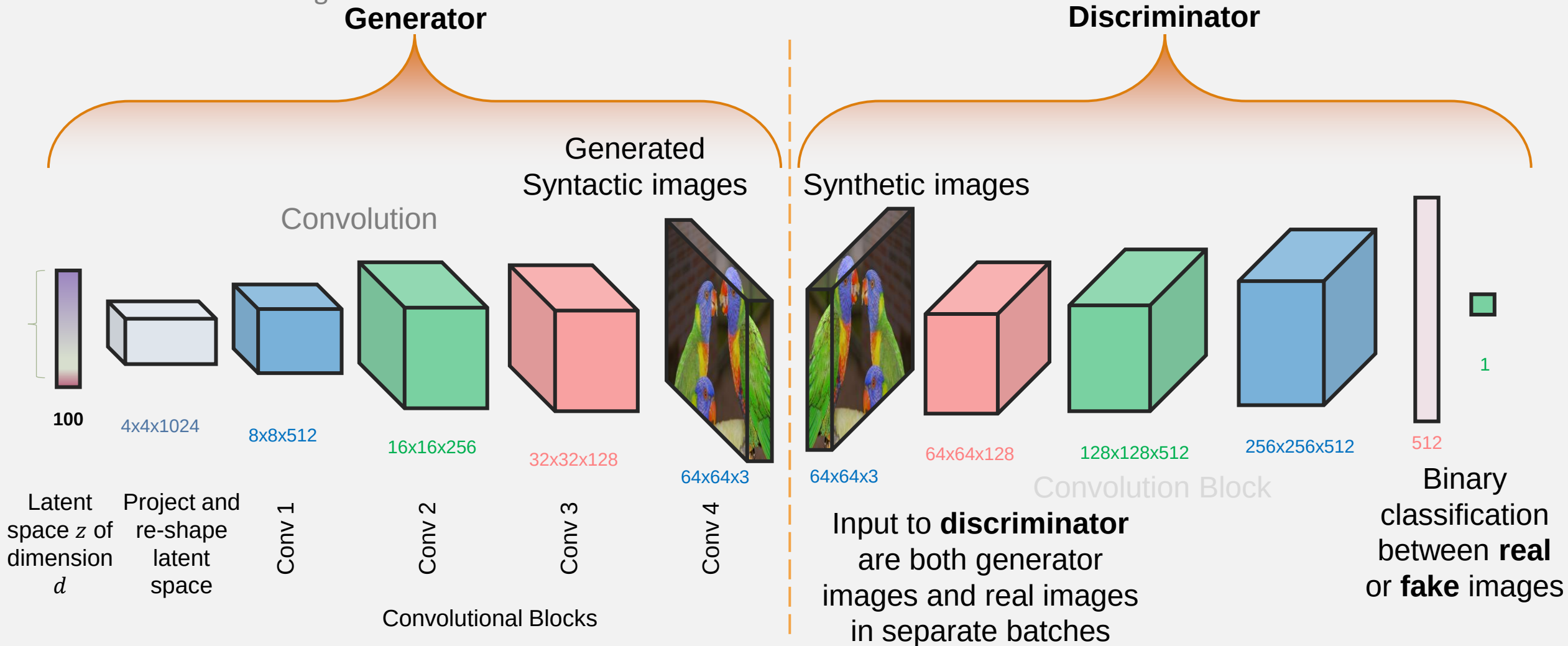
$$\min_G V(D, G) = \mathbb{E}_z [\log(1 - D(G(z)))]$$



Deep Convolutional GAN (DCGAN)

3:33 PM

The task of the generator is to produce data which the discriminator predicts as being 'real', meaning that it closely resembles the training dataset.



Super Realistic Generative Adversarial Networks

bicubic
(21.59dB/0.6423)



SRResNet
(23.53dB/0.7832)



SRGAN
(21.15dB/0.6868)



original



StyleGAN to Generate China City Scape

3:33 PM

One of my student's project



(Jia 2022)

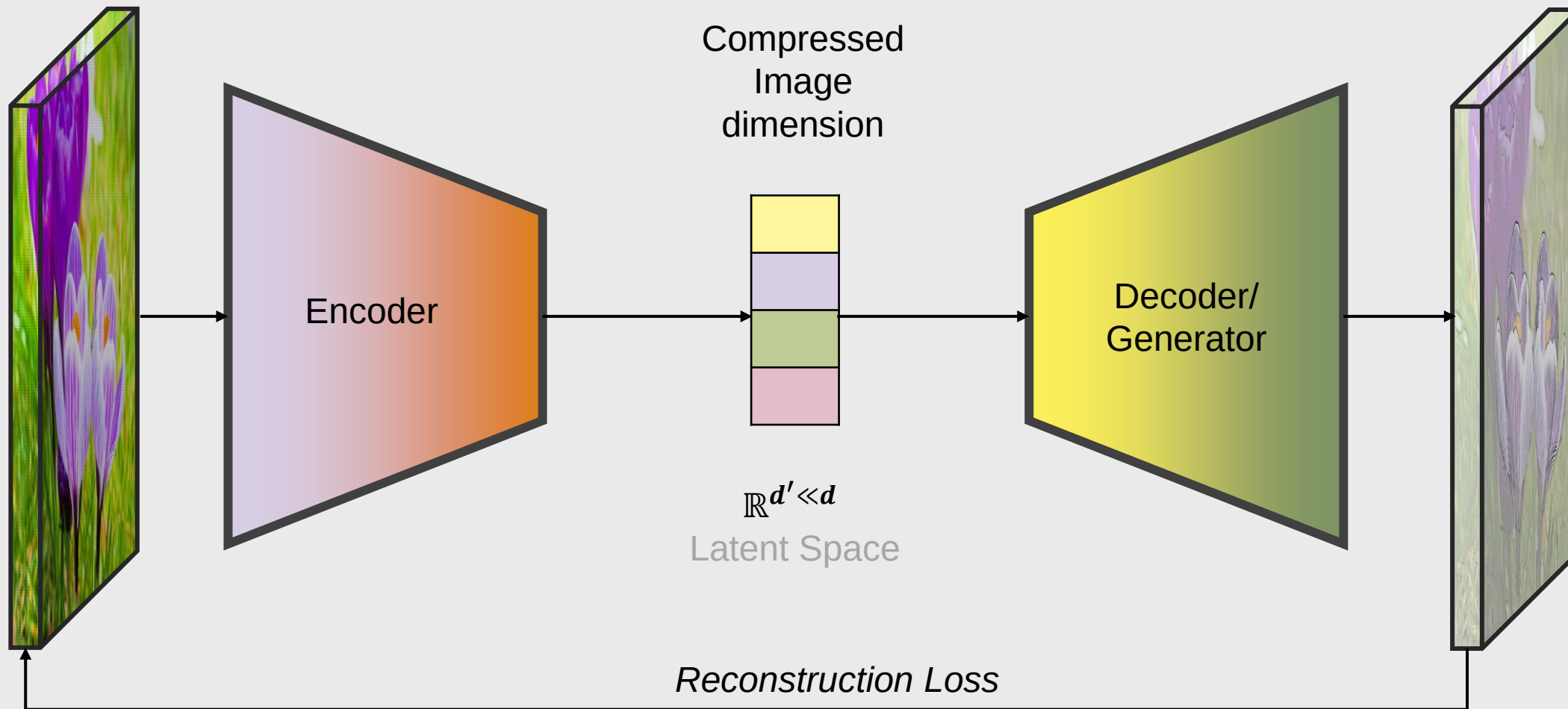
Autoencoder

3:33 PM

Can be used for dimensionality reduction as well.

Input
Image $\mathbf{x} \in \mathbb{R}^d$

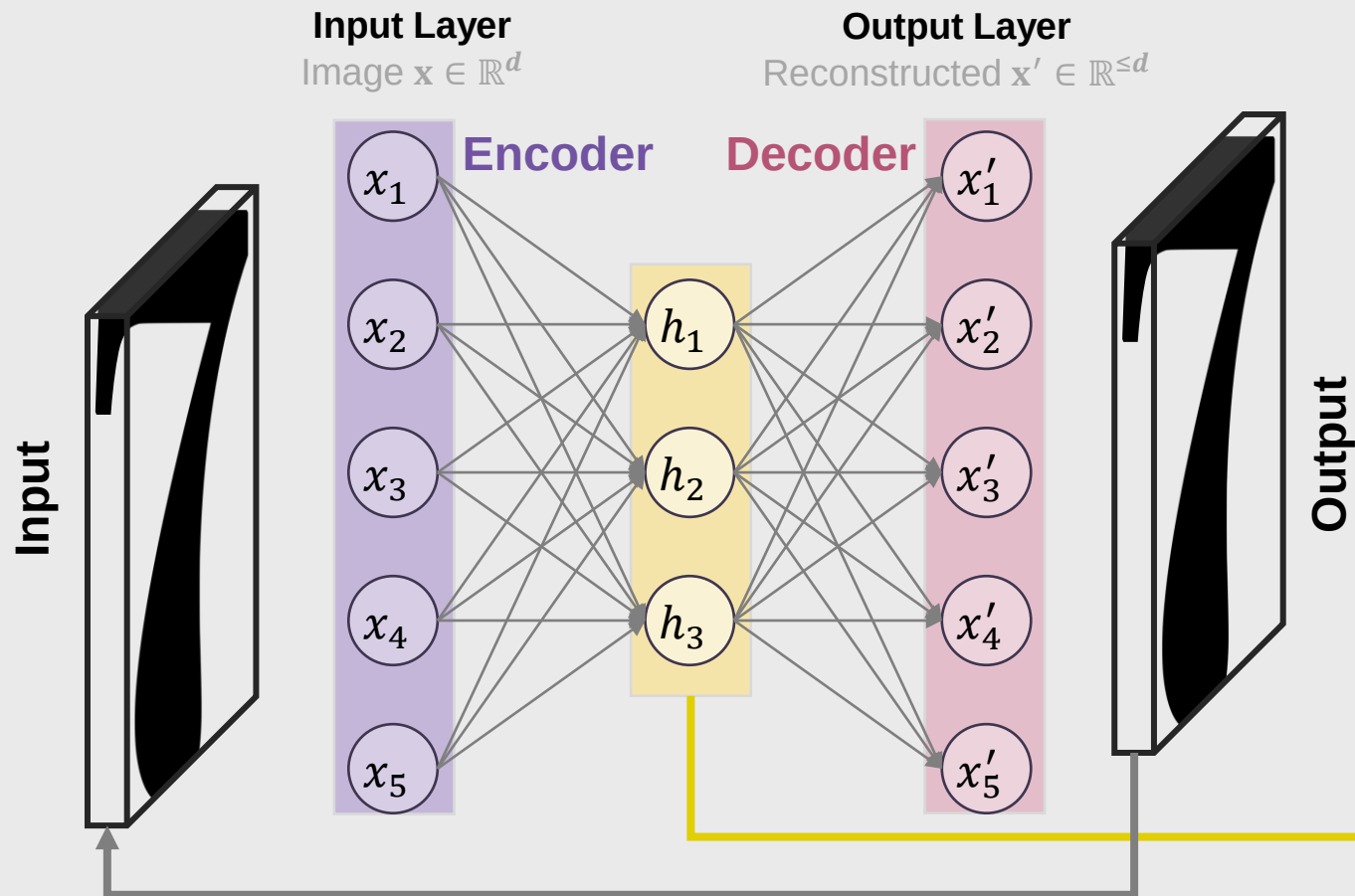
Reconstructed
Image $\mathbf{x}' \in \mathbb{R}^{\leq d}$



Autoencoder

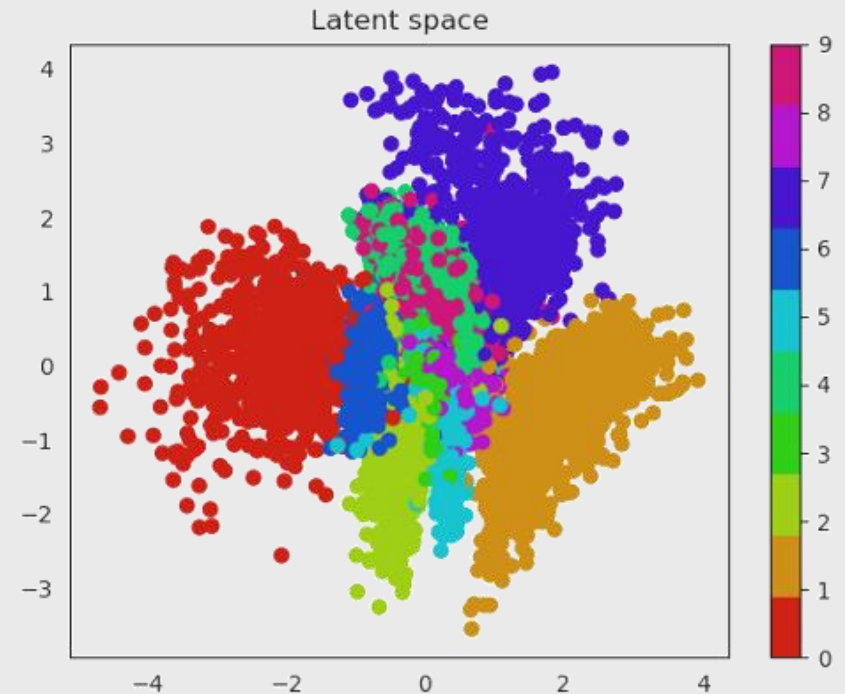
3:33 PM

Autoencoder takes its feedback/error correction signal from the input itself This it is **Unsupervised Or Self-Supervised**



Reconstruction Loss (e.g., Mean Squared Error)

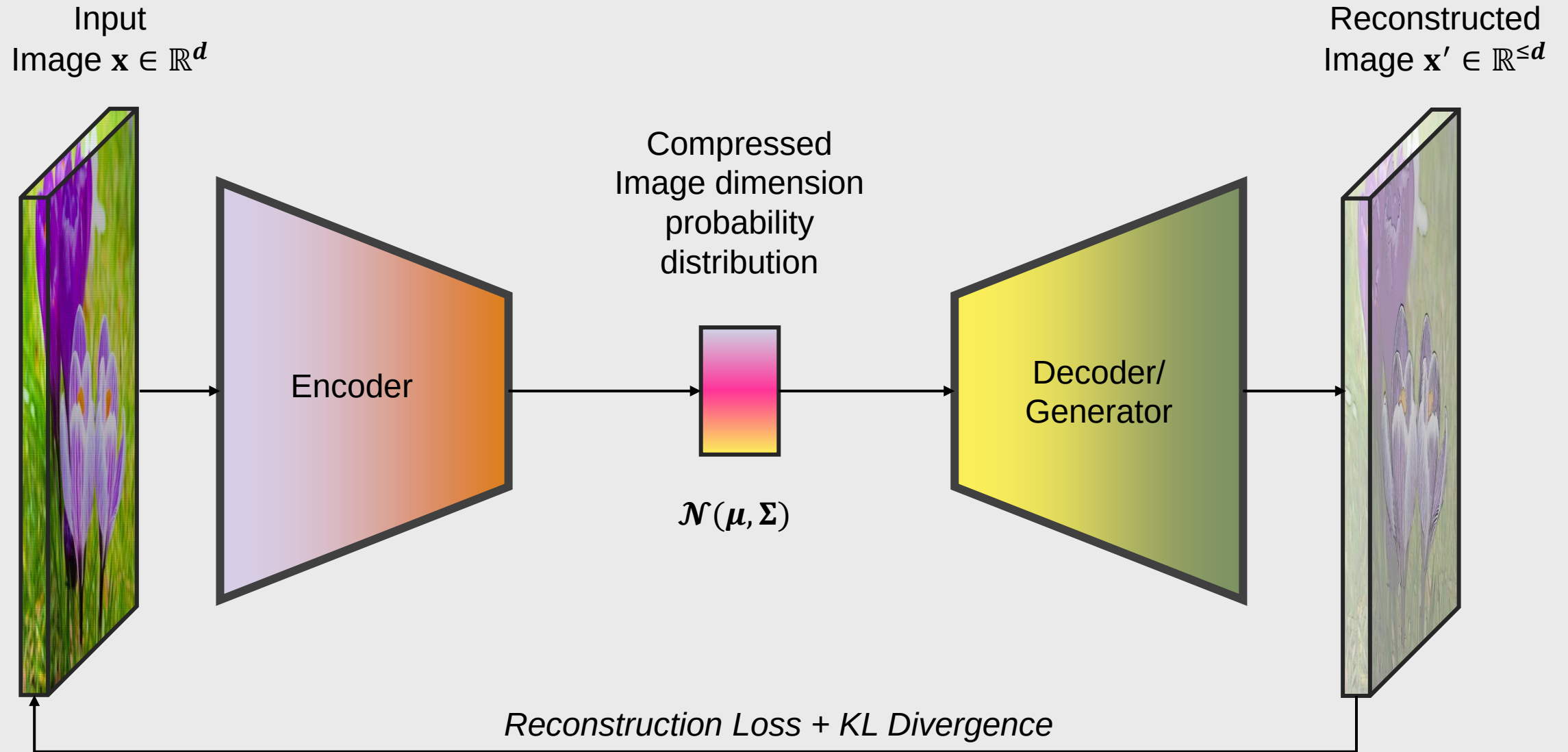
$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (x_i - x'_i)^2$$



Latent space give compressed feature representation of input. But it give no guarantee that the hidden units extract meaningful feature.

Variational Autoencoder

3:33 PM



Auto-Encoding Variational Bayes

Diederik P. Kingma

Machine Learning Group
Universiteit van Amsterdam
dpkingma@gmail.com

Max Welling

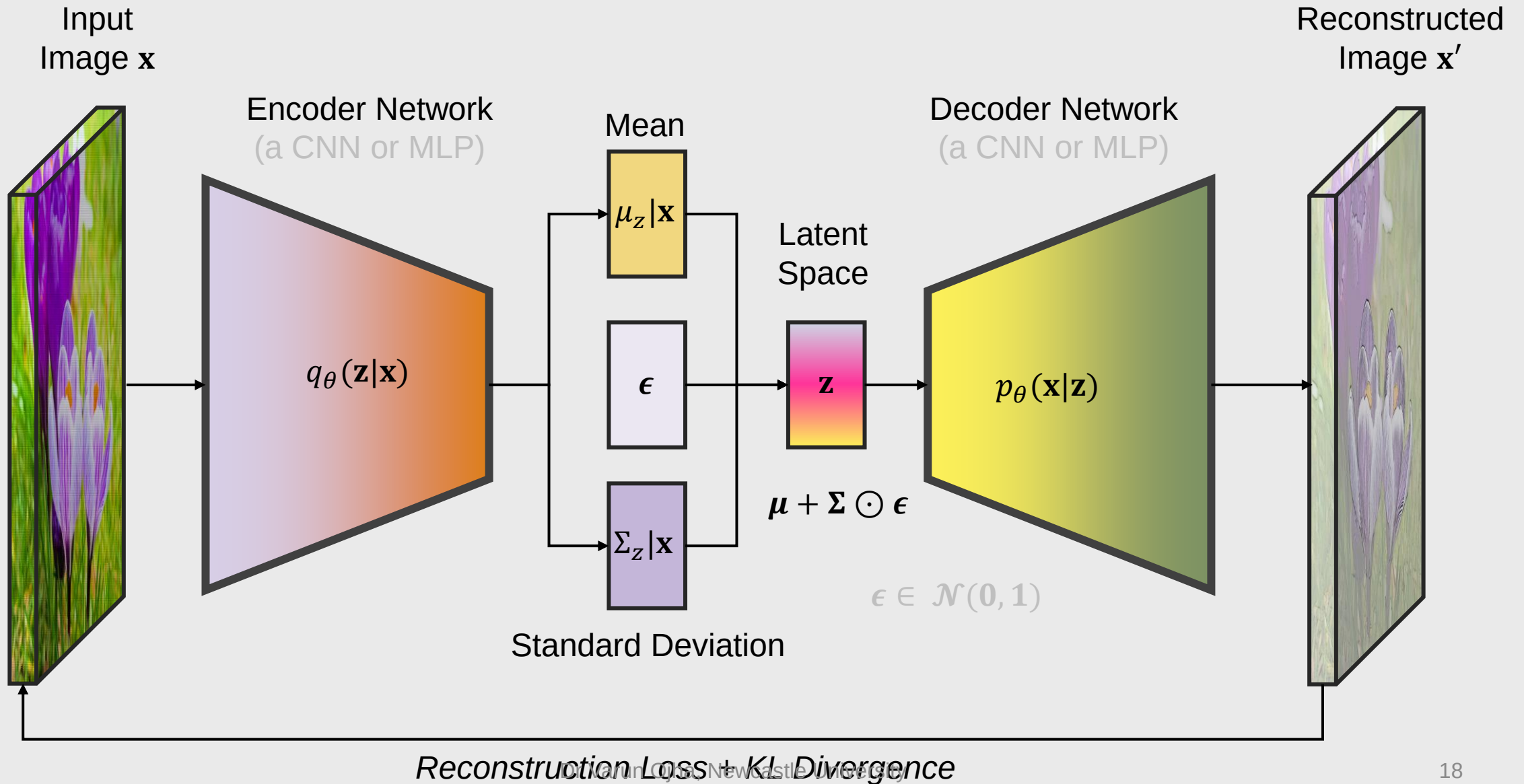
Machine Learning Group
Universiteit van Amsterdam
welling.max@gmail.com

Abstract

How can we perform efficient inference and learning in directed probabilistic models, in the presence of continuous latent variables with intractable posterior distributions, and large datasets? We introduce a stochastic variational inference and learning algorithm that scales to large datasets and, under some mild differentiability conditions, even works in the intractable case. Our contributions is two-fold. First, we show that a reparameterization of the variational lower bound yields a lower bound estimator that can be straightforwardly optimized using standard stochastic gradient methods. Second, we show that for i.i.d. datasets with continuous latent variables per datapoint, posterior inference can be made especially efficient by fitting an approximate inference model (also called a recognition model) to the intractable posterior using the proposed lower bound estimator. Theoretical advantages are reflected in experimental results.

Variational Autoencoder

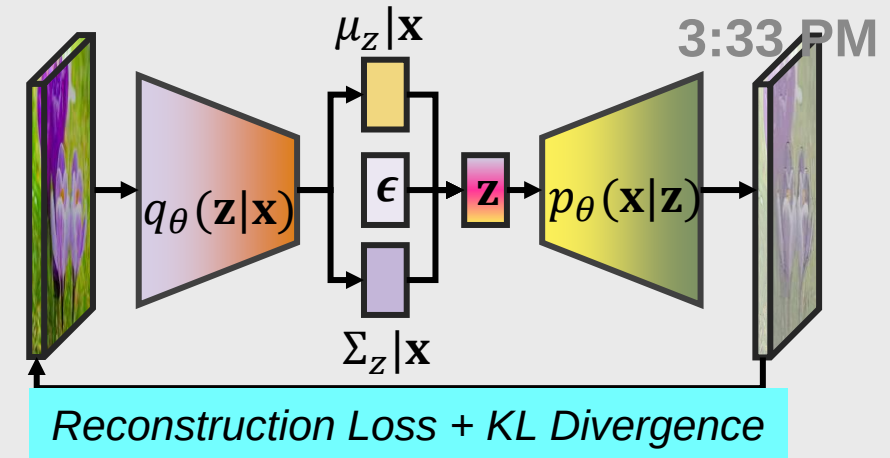
3:33 PM



Loss function of VAE

VAE Loss = Reconstruction Loss + Regularisation Loss

$$L_{VAE} = L_{Reconstruction} + L_{Regularisation}$$



Reconstruction Loss

It answers how well the decoder reconstructs the input from the latent sample.

- **Mean Squared Error (MSE)** for continuous data and
- **Cross Entropy (CE)** categorical data (e.g., MNIST)

$$L_{Reconstruction} = \mathbb{E}_{q(z|x)} [-\log p(x|z)]$$

Regularization Loss

It answers how far the learned latent distribution $q(z|x)$ is from the desired prior distribution $p(z) = N(0, 1)$

$$L_{Regularisation} = D_{KL}(q(z|x) || p(z))$$

Mean Squared Error

5	3
1	6

Image A

2	4
1	3

Image B

$(5 - 2)^2$	$(3 - 4)^2$
$(1 - 1)^2$	$(6 - 3)^2$

Pixelwise squared difference

9	+	1	+
0	+	9	=

Pixelwise sum

4.5

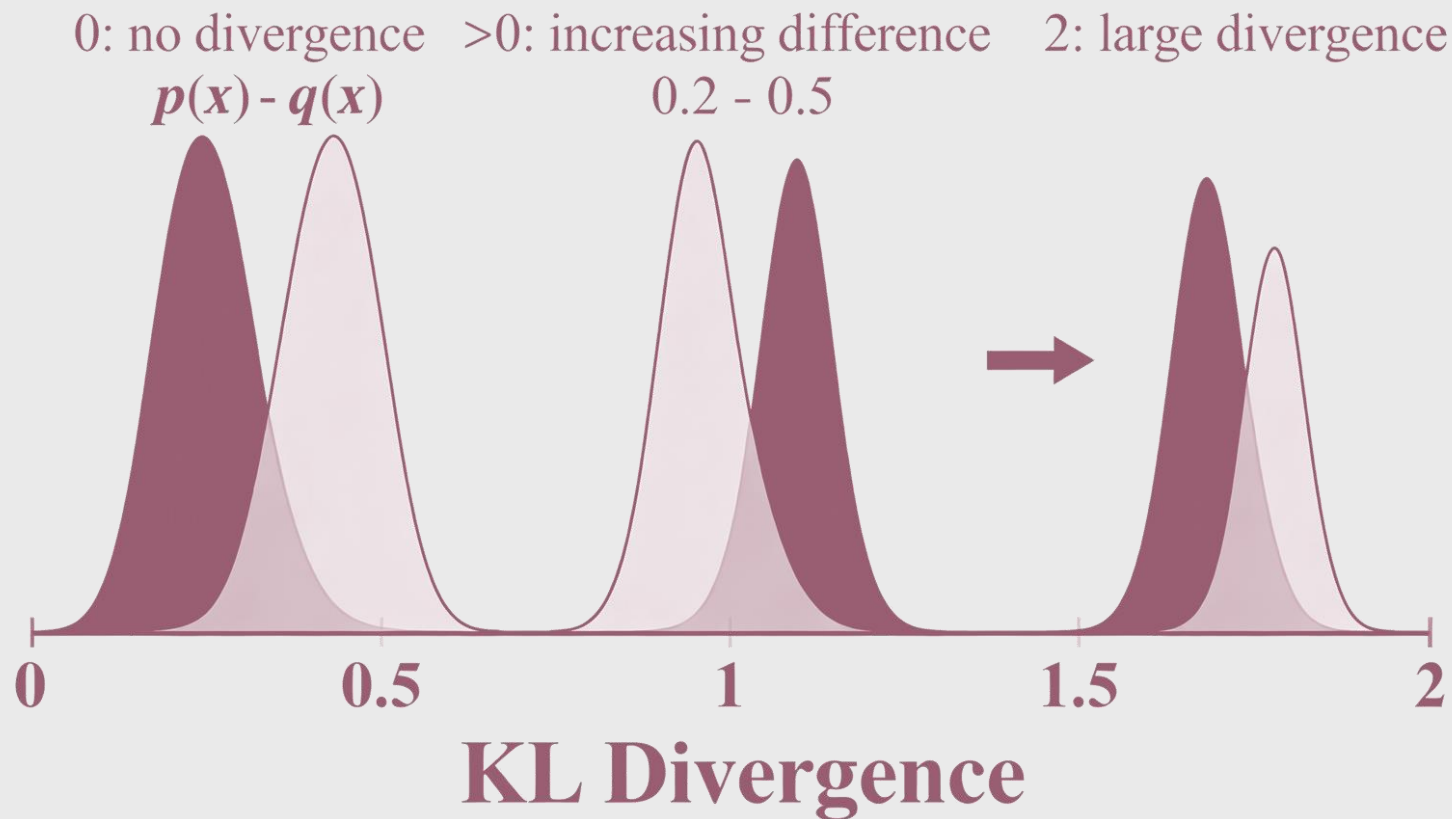
Mean

$$MSE = \frac{1}{N} \sum_{ij}^N (A_{ij} - B_{ij})^2$$

where N is the dimension of a square Metrix/Image

Kullback–Leibler (KL) divergence

KL Divergence measure distance between two probability distributions (much like distance between two points)



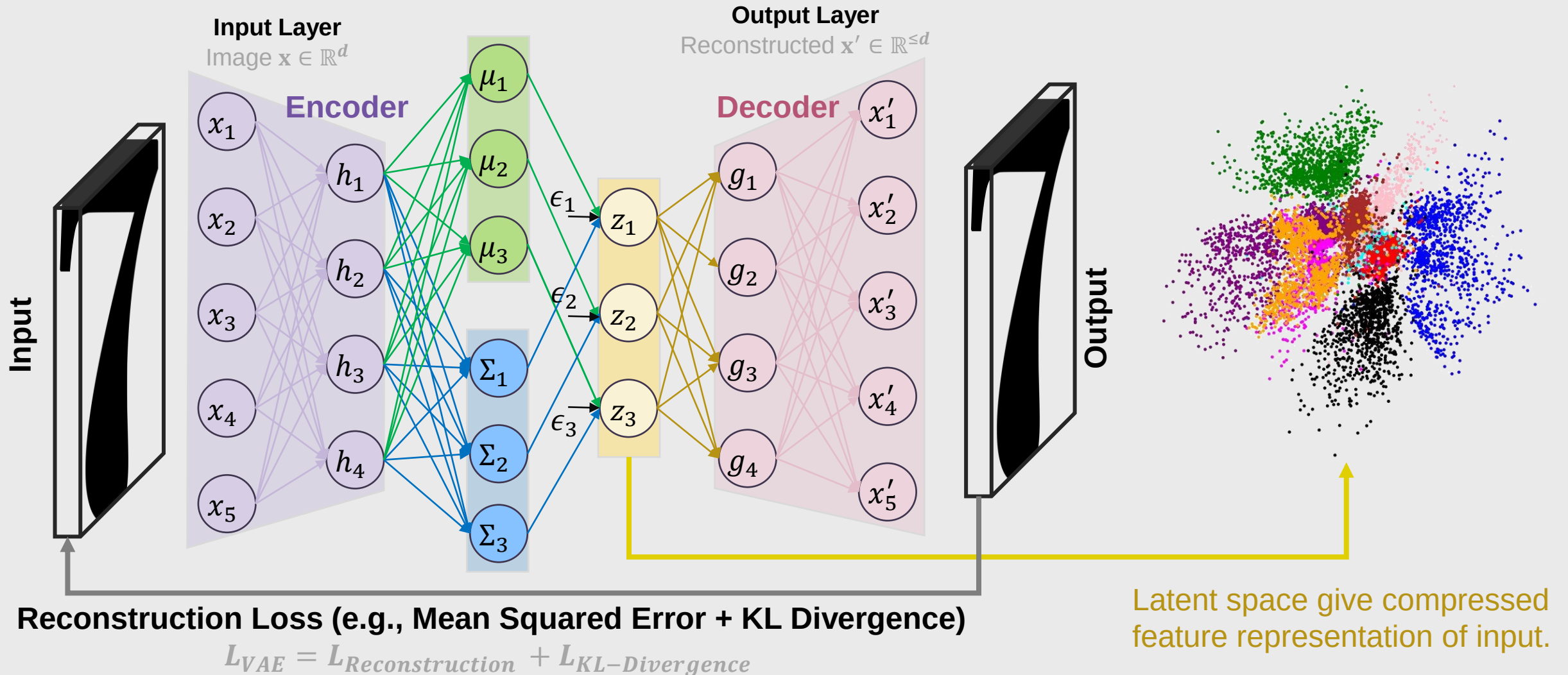
$$D_{KL} = \sum_{x \in X} p(x) \log \left(\frac{p(x)}{q(x)} \right)$$

The lower bound value is zero and is achieved when the distributions under observation are identical.

Variational Autoencoder

3:33 PM

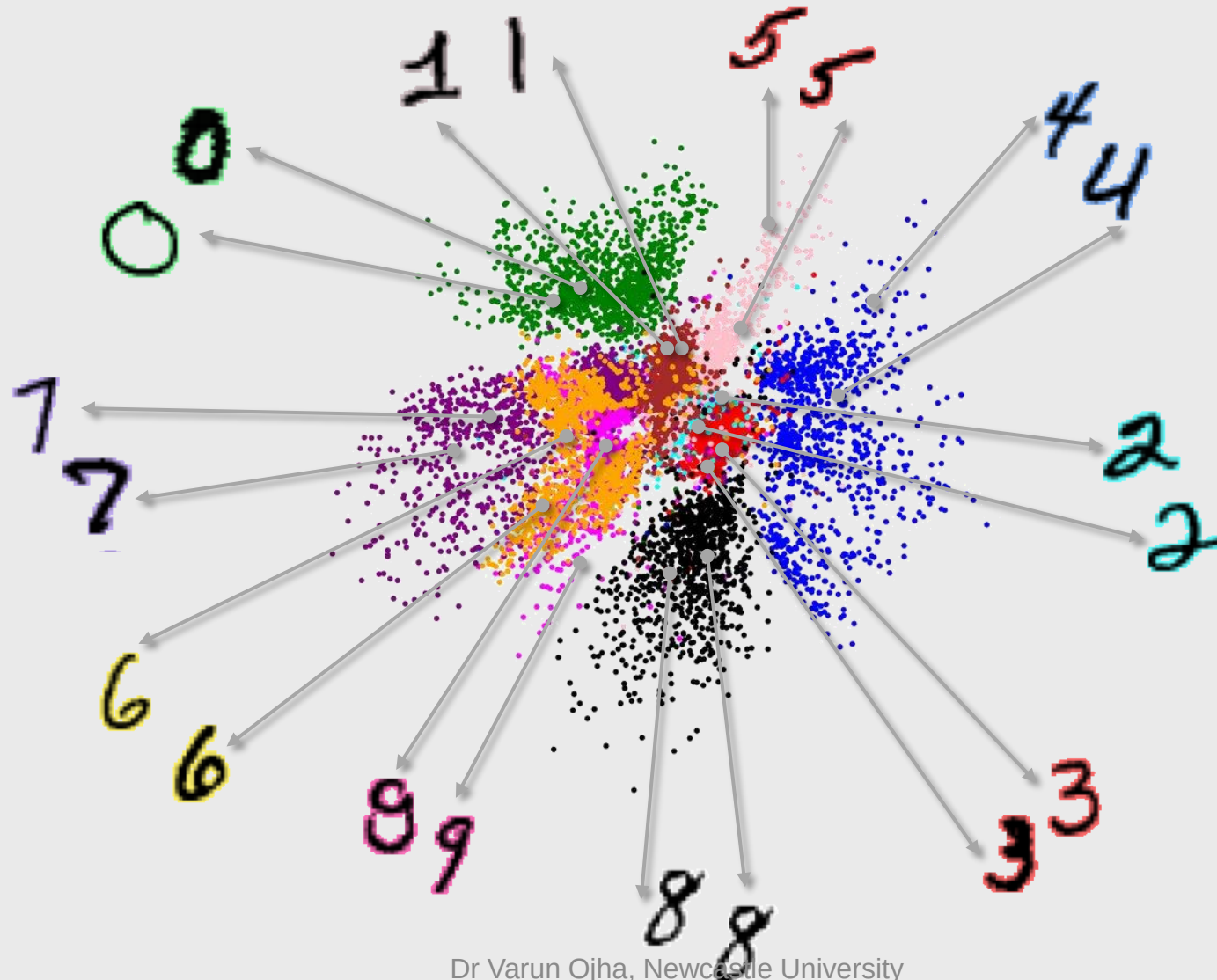
Autoencoder takes its feedback/error correction signal from the input itself. This is **Unsupervised Or Self-Supervised**



Variational Autoencoder

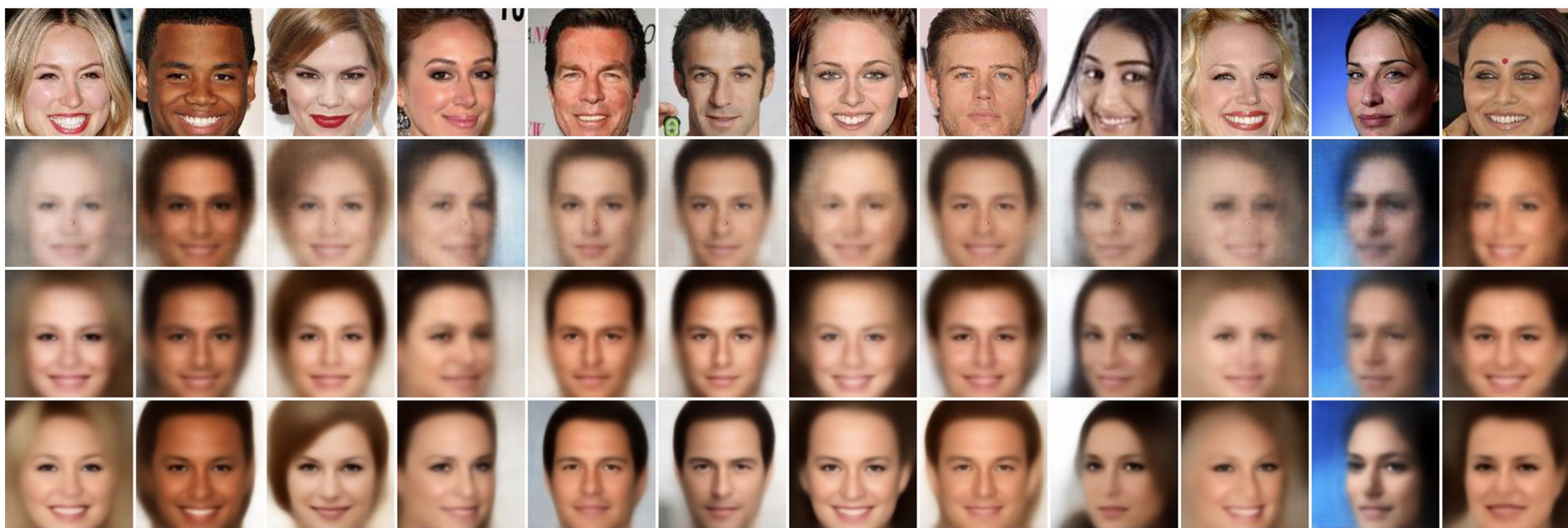
3:33 PM

Latent space give compressed feature representation of input.



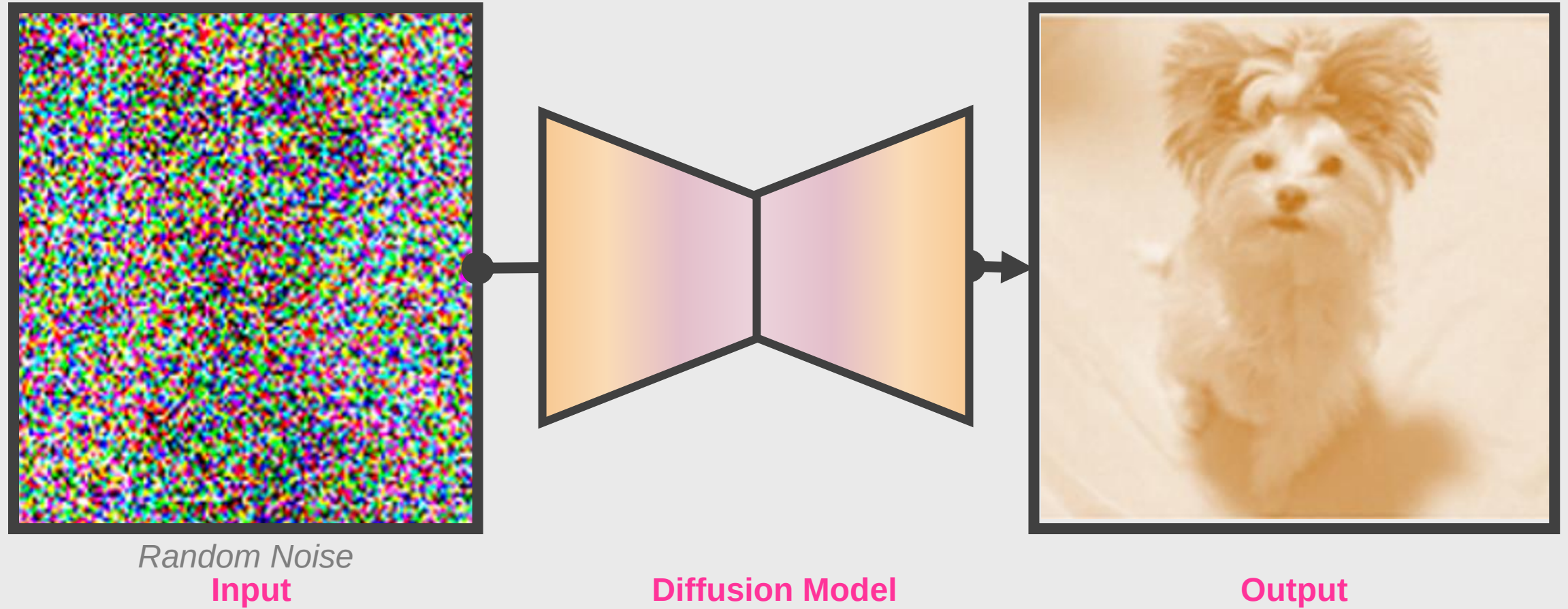
Reconstructed Images of Celeb Face Image

3:33 PM



Cai et al (2017) Multi-Stage Variational Auto-Encoders for Coarse-to-Fine Image Generation: <https://arxiv.org/abs/1705.07202>

Diffusion Models



Diffusion Models

forward diffusion



reverse diffusion

Diffusion

3:33 PM

random molecular movement cause molecule to spread out all over, or *diffuse*, over space

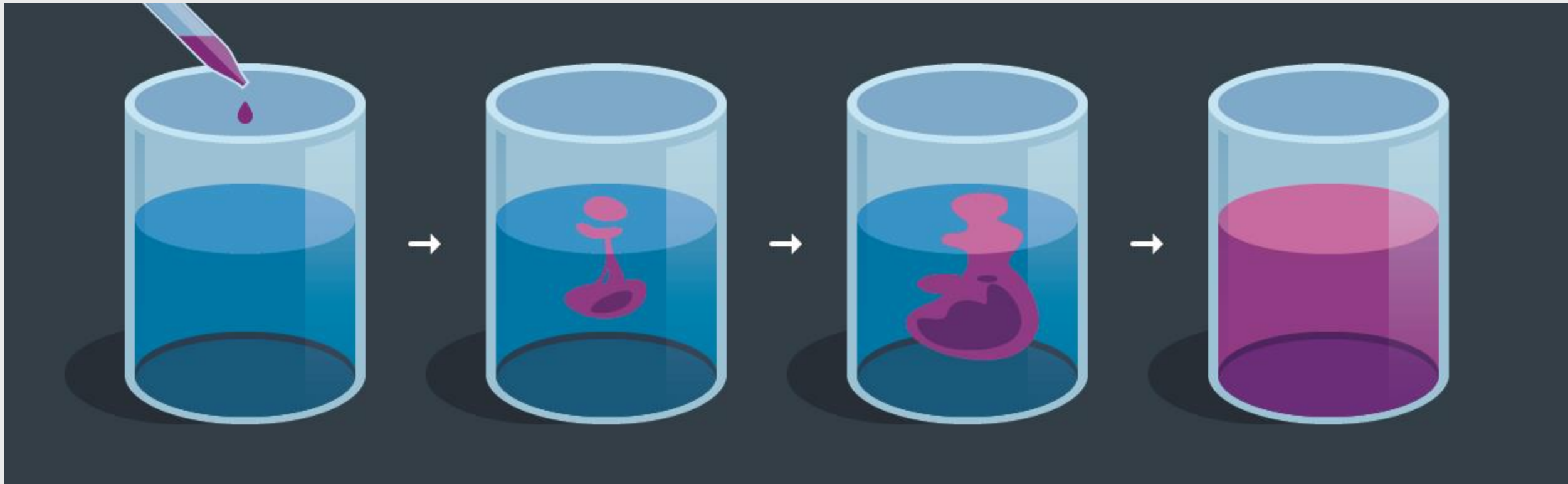


image credit: Visionlearning

Diffusion Models

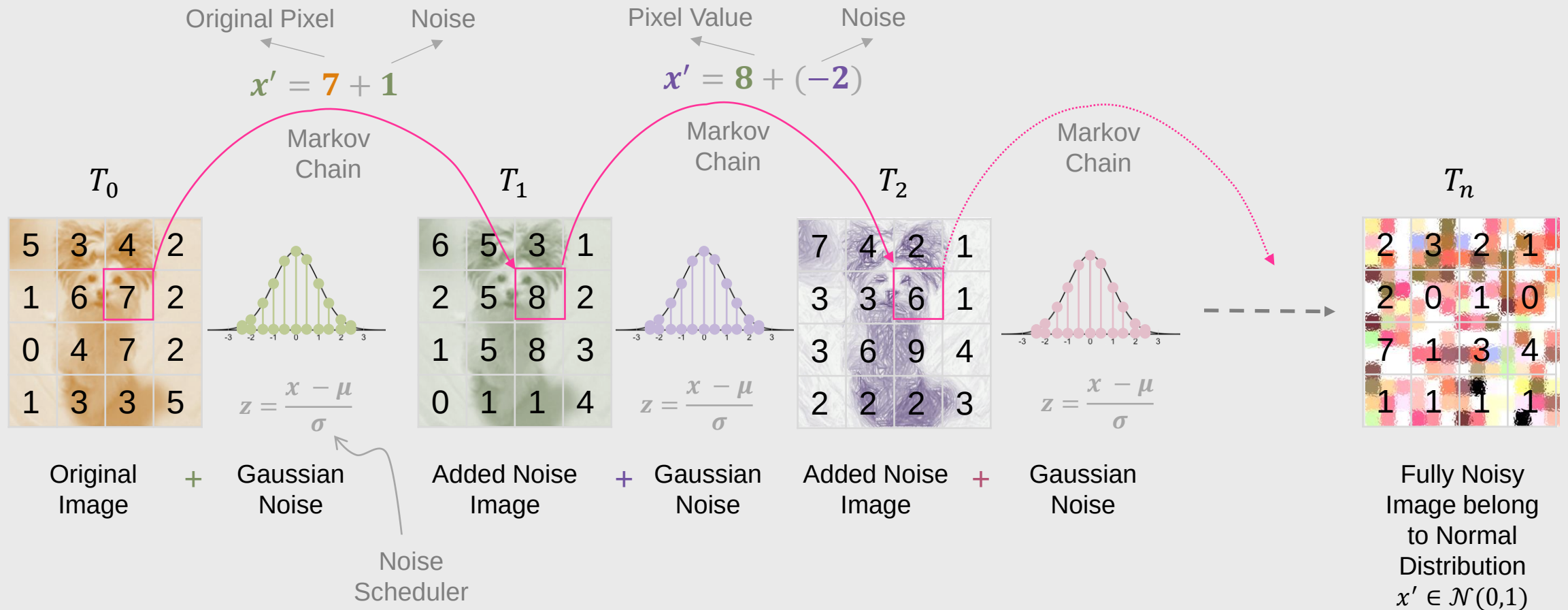
forward diffusion



reverse diffusion

Forward Diffusion

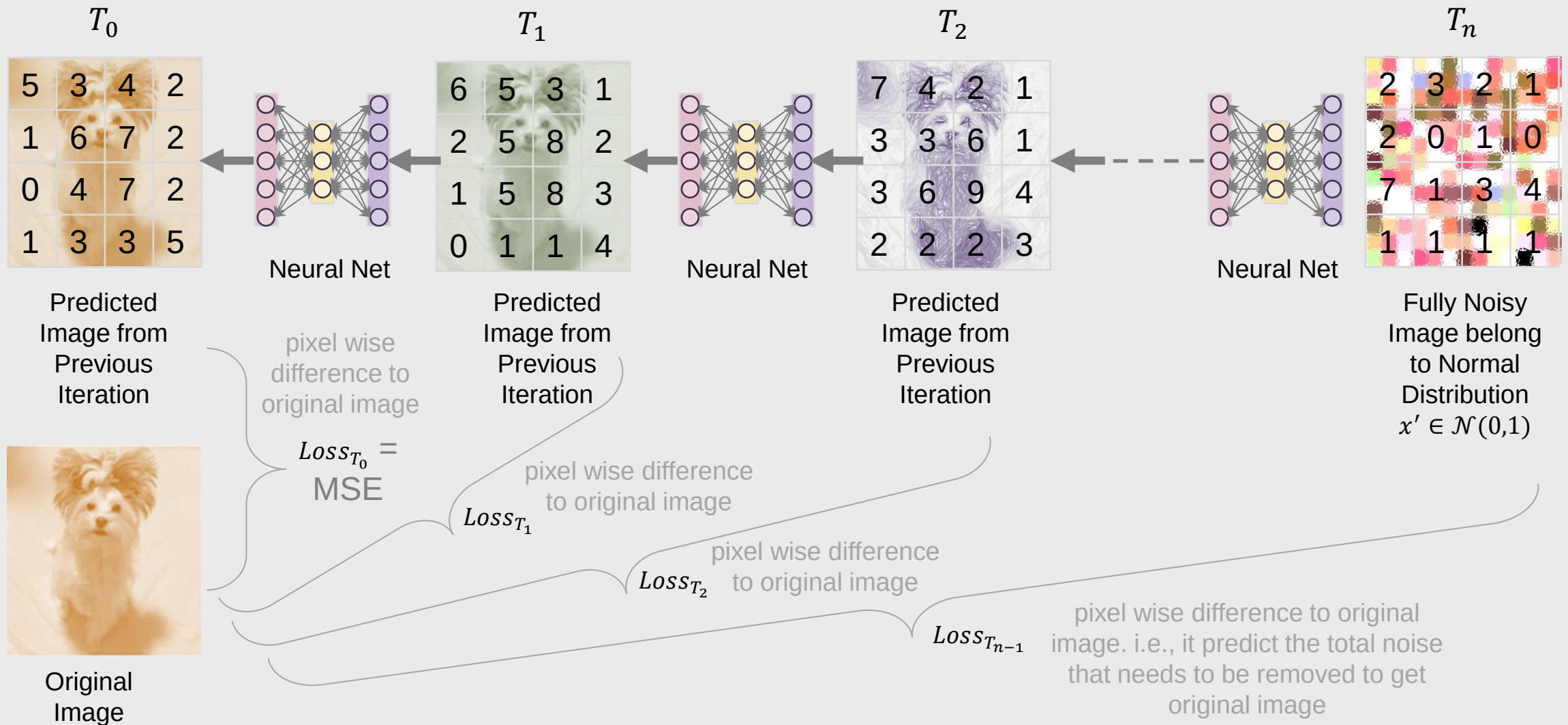
3:33 PM

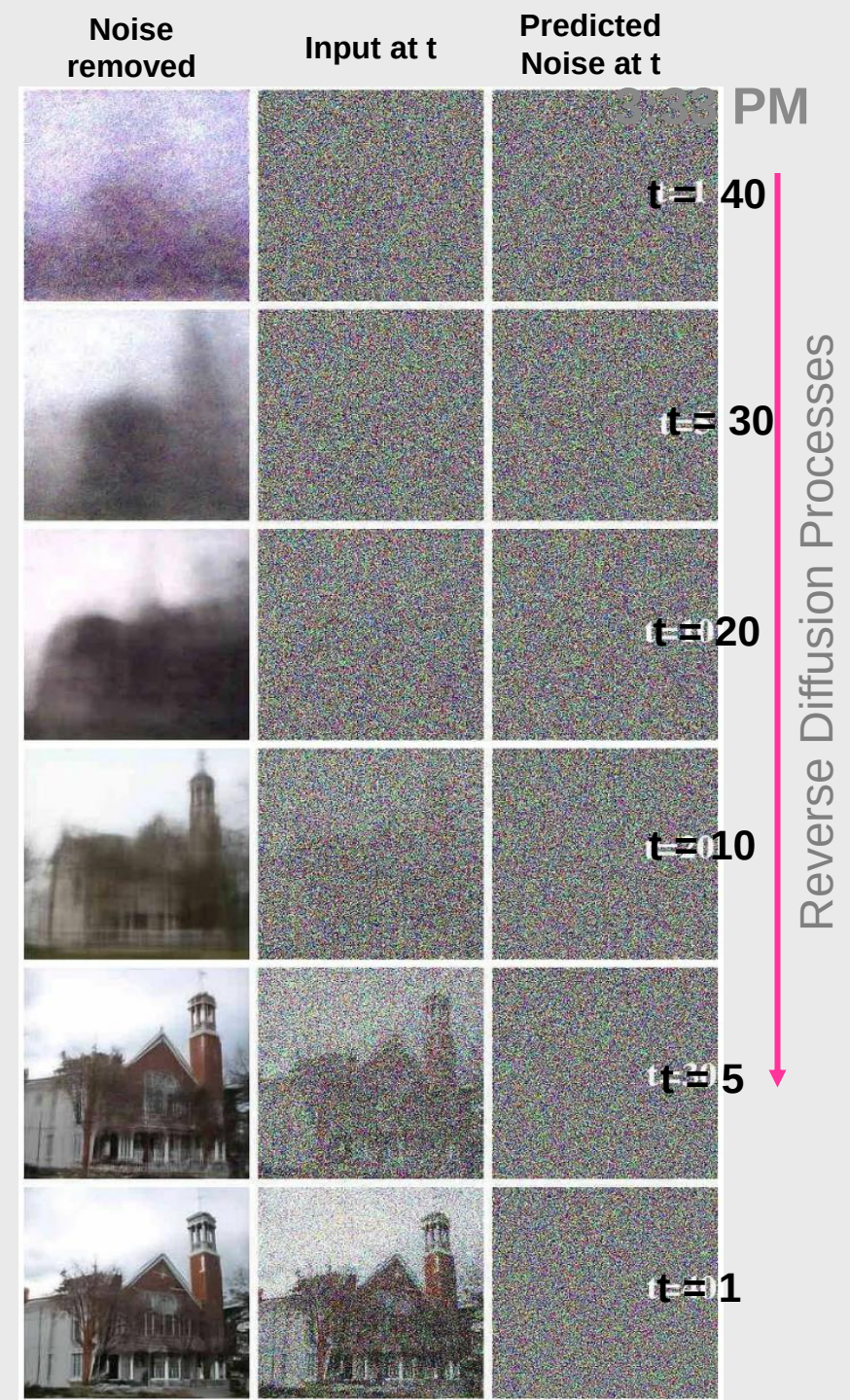
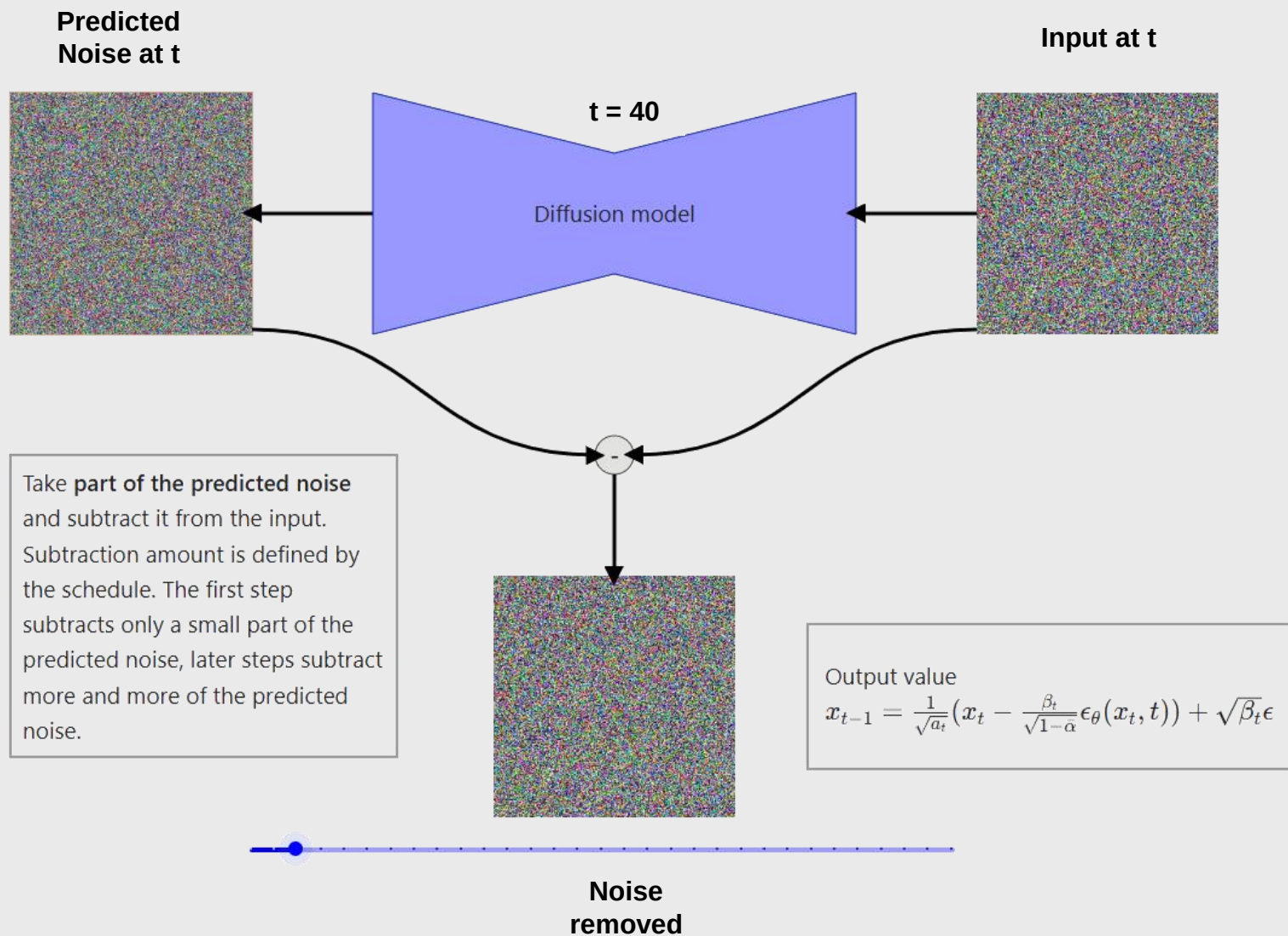


Images generated in each successive steps are used as training data in reverse diffusion

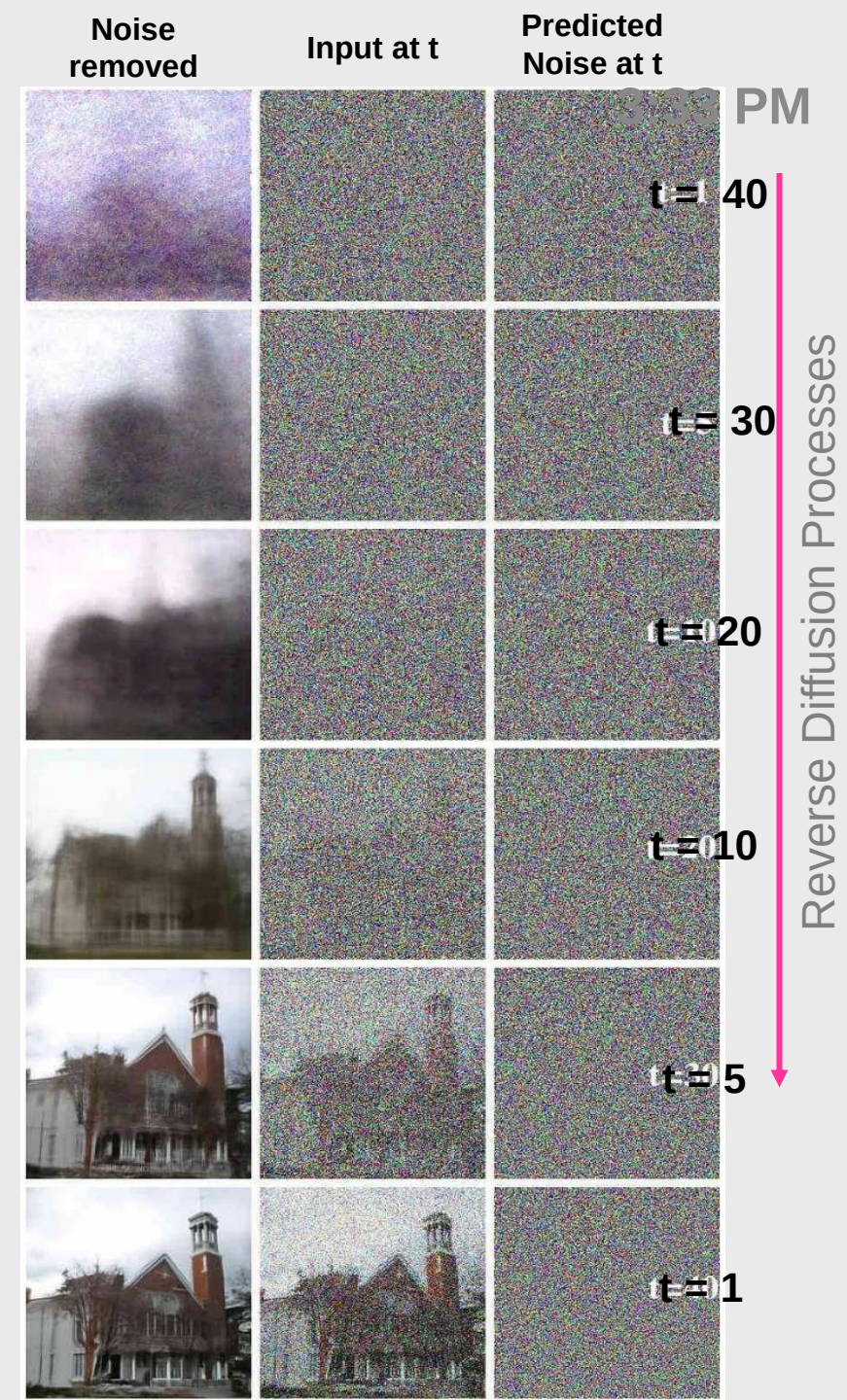
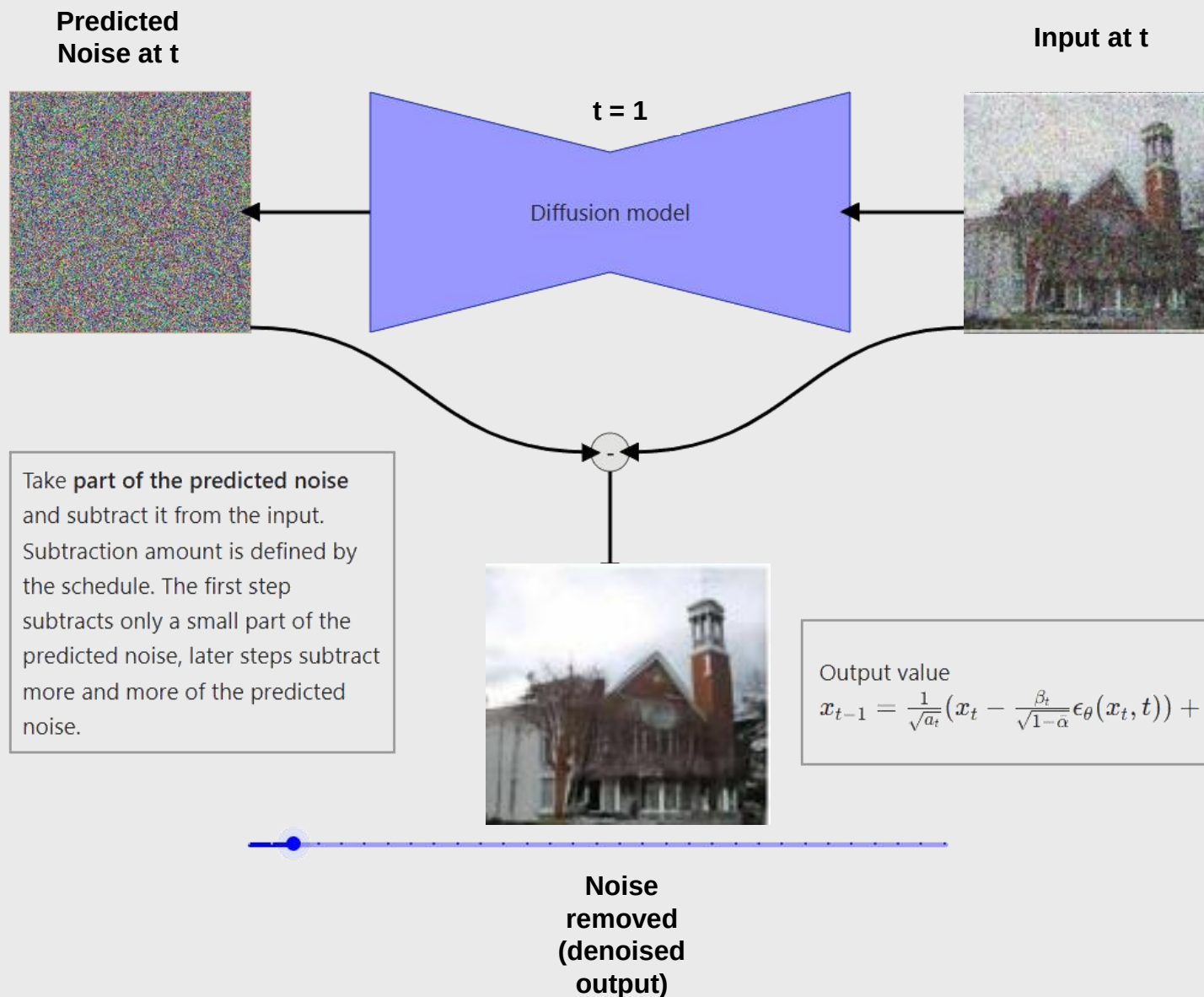
Reverse Diffusion

3:33 PM





Source & Visual Illustration: <https://erdem.pl/2023/11/step-by-step-visual-introduction-to-diffusion-models#reverse-diffusion-diagram>

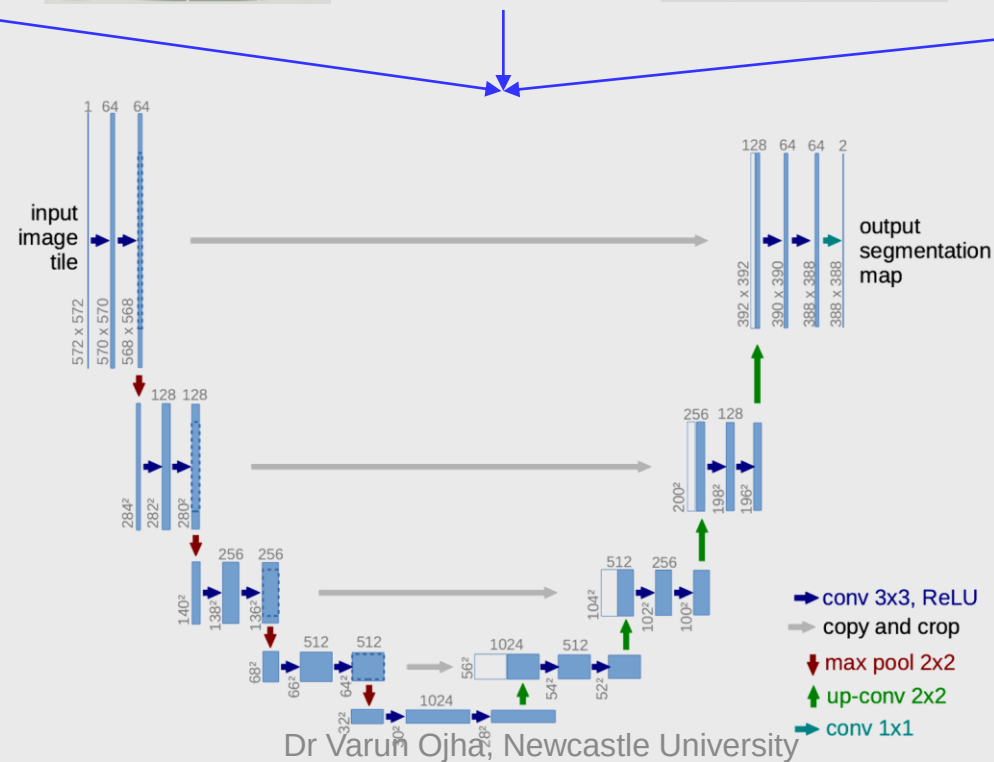


Source & Visual Illustration: <https://erdem.pl/2023/11/step-by-step-visual-introduction-to-diffusion-models#reverse-diffusion-diagram>

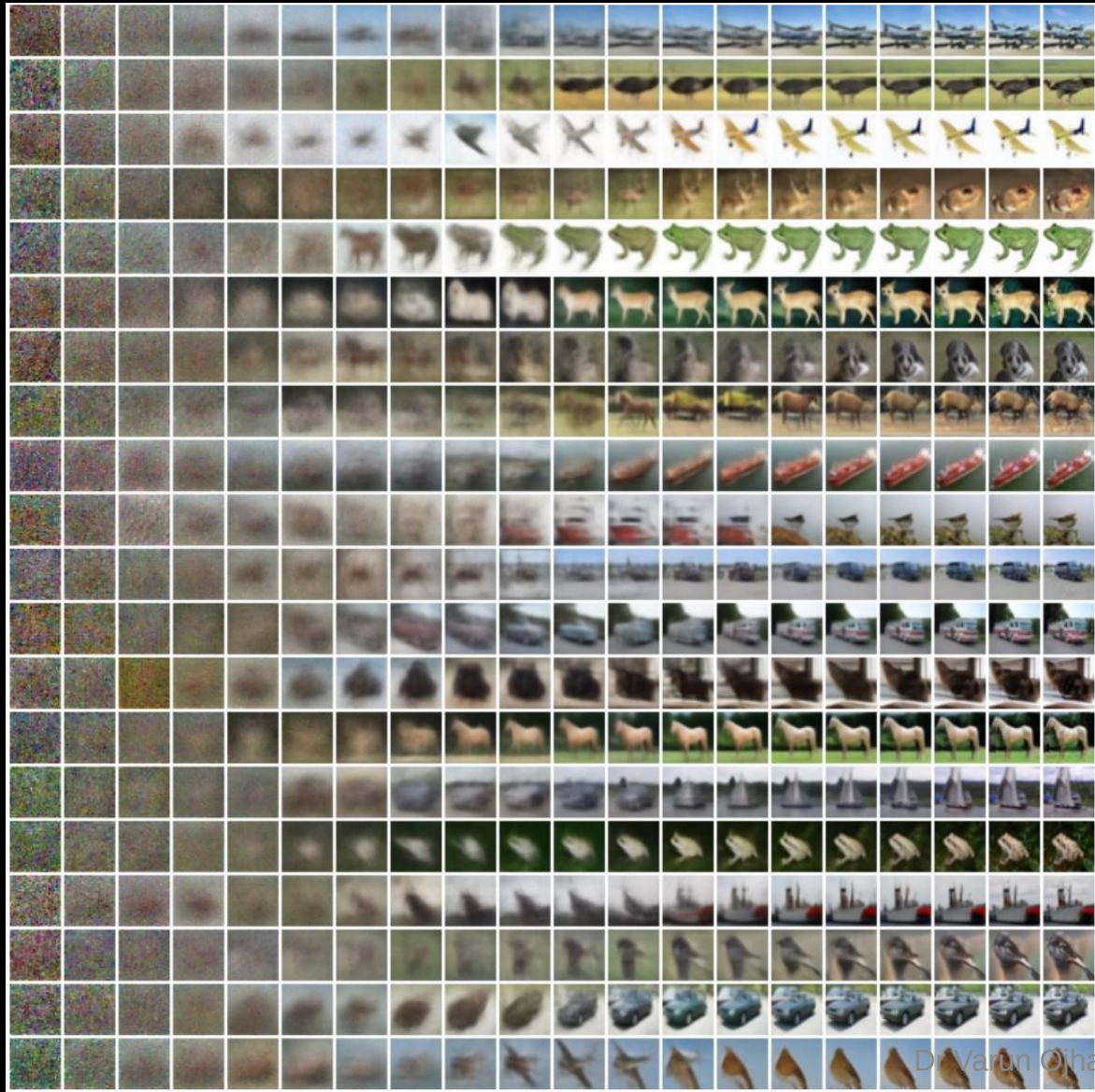
3:33 PM

The diagram illustrates the iterative process of the proposed method. It shows a sequence of time steps $T_0, T_1, T_2, \dots, T_n$. At each step, a 4x4 grid of numbers is processed by a "Neural Net" (represented by a diagram of a neural network) to produce the next grid. The grids show a progression of values, with some cells containing numbers and others being blank or showing a different pattern. The final grid T_n shows a more complex, noisy pattern of numbers.

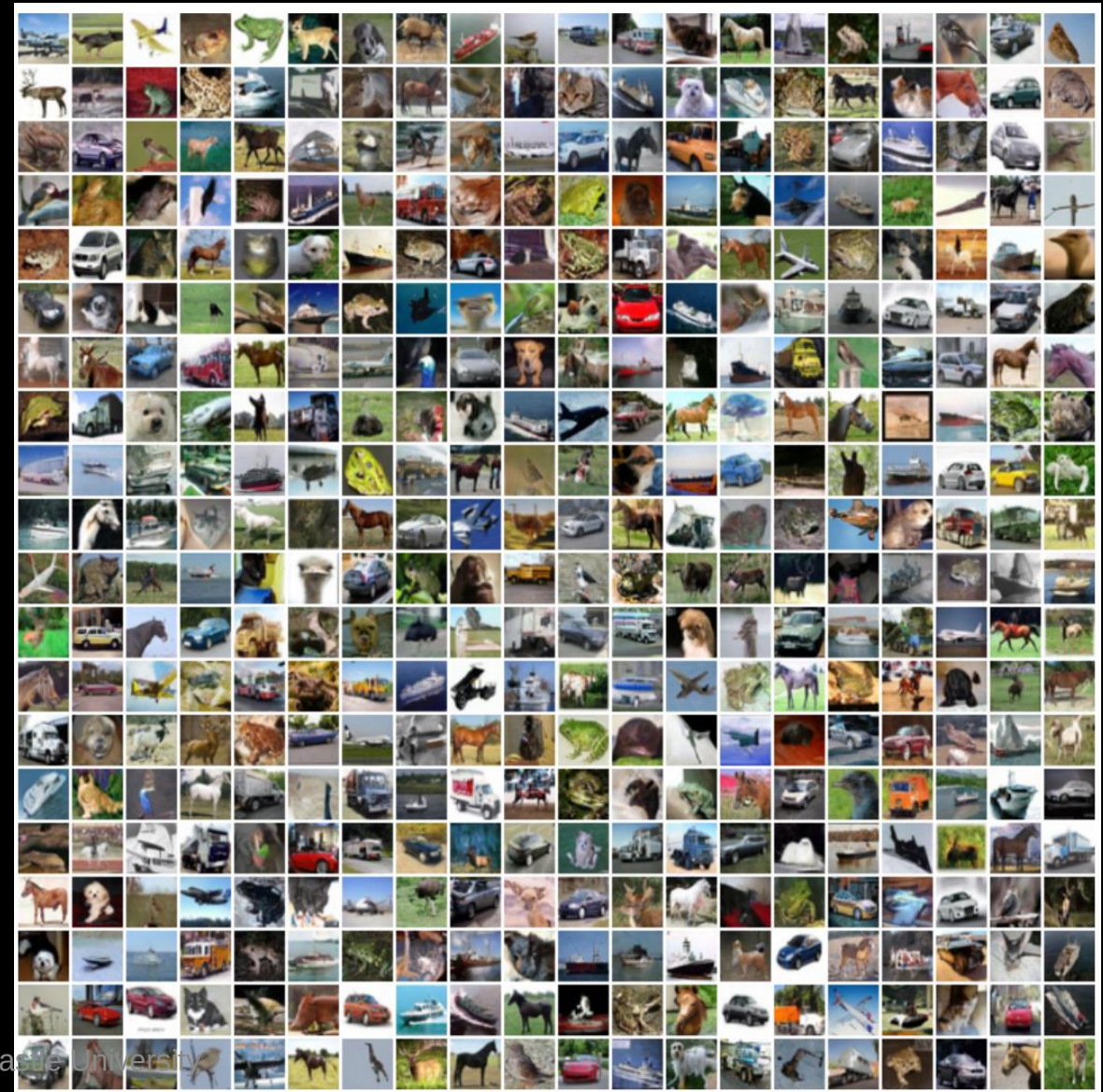
A U-Net architecture is used as the backbone in diffusion models. A single shared U-Net is trained on success pairs of images (T_i, T_{i-1}) at different diffusion steps



Sample Generation Processes



Generated Samples



3:33 PM

Jonathan Ho UC Berkeley jonathanho@berkeley.edu
 Ajay Jain UC Berkeley ajayj@berkeley.edu
 Pieter Abbeel UC Berkeley pabbeel@cs.berkeley.edu

Abstract

We present high quality image synthesis results using diffusion probabilistic models, a class of latent variable models inspired by considerations from nonequilibrium thermodynamics. Our best results are obtained by training on a weighted variational bound designed according to a novel connection between diffusion probabilistic models and denoising score matching with Langevin dynamics, and our models naturally admit a progressive lossy decompression scheme that can be interpreted as a generalization of autoregressive decoding. On the unconditional CIFAR10 dataset, we obtain an Inception score of 9.46 and a state-of-the-art FID score of 3.17. On 256x256 LSUN, we obtain sample quality similar to ProgressiveGAN. Our implementation is available at <https://github.com/hojonathanho/diffusion>.

1 Introduction

Deep generative models of all kinds have recently exhibited high quality samples in a wide variety of data modalities. Generative adversarial networks (GANs), autoregressive models, flows, and variational autoencoders (VAEs) have synthesized striking image and audio samples [14, 27, 3, 58, 38, 25, 10, 32, 44, 57, 26, 33, 45], and there have been remarkable advances in energy-based modeling and score matching that have produced images comparable to those of GANs [11, 55].

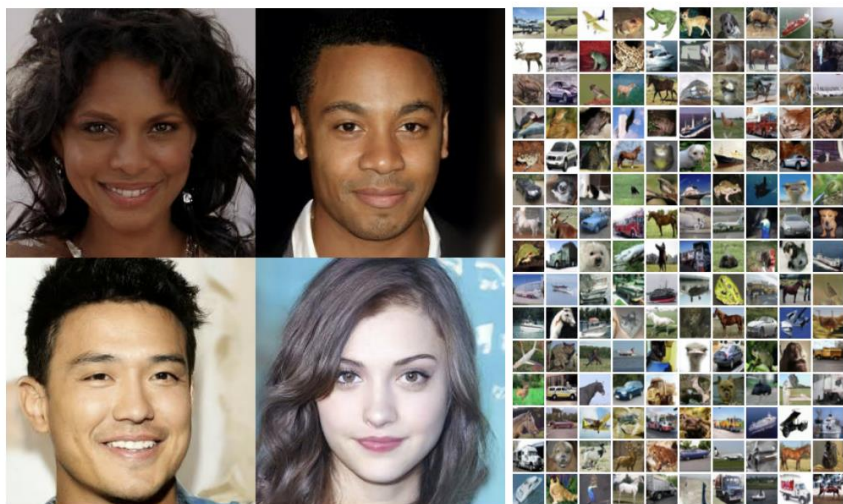


Figure 1: Generated samples on CelebA-HQ 256 × 256 (left) and unconditional CIFAR10 (right)

Jascha Sohl-Dickstein
 Stanford University

JASCHA@STANFORD.EDU

Eric A. Weiss
 University of California, Berkeley

EWEISS@BERKELEY.EDU

Niru Maheswaranathan
 Stanford University

NIRUM@STANFORD.EDU

Surya Ganguli
 Stanford University

SGANGULI@STANFORD.EDU

Abstract

A central problem in machine learning involves modeling complex data-sets using highly flexible families of probability distributions in which learning, sampling, inference, and evaluation are still analytically or computationally tractable. Here, we develop an approach that simultaneously achieves both flexibility and tractability. The essential idea, inspired by non-equilibrium statistical physics, is to systematically and slowly destroy structure in a data distribution through an iterative forward diffusion process. We then learn a reverse diffusion process that restores structure in data, yielding a highly flexible and tractable generative model of the data. This approach allows us to rapidly learn, sample from, and evaluate probabilities in deep generative models with thousands of layers or time steps, as well as to compute conditional and posterior probabilities under the learned model. We additionally release an open source reference implementation of the algorithm.

1. Introduction

Historically, probabilistic models suffer from a tradeoff between two conflicting objectives: *tractability* and *flexibility*. Models that are *tractable* can be analytically evaluated and easily fit to data (e.g. a Gaussian or Laplace). However,

these models are unable to aptly describe structure in rich datasets. On the other hand, models that are *flexible* can be molded to fit structure in arbitrary data. For example, we can define models in terms of any (non-negative) function $\phi(\mathbf{x})$ yielding the flexible distribution $p(\mathbf{x}) = \frac{\phi(\mathbf{x})}{Z}$, where Z is a normalization constant. However, computing this normalization constant is generally intractable. Evaluating, training, or drawing samples from such flexible models typically requires a very expensive Monte Carlo process.

A variety of analytic approximations exist which ameliorate, but do not remove, this tradeoff—for instance mean field theory and its expansions (T, 1982; Tanaka, 1998), variational Bayes (Jordan et al., 1999), contrastive divergence (Welling & Hinton, 2002; Hinton, 2002), minimum probability flow (Sohl-Dickstein et al., 2011b;a), minimum KL contraction (Lyu, 2011), proper scoring rules (Gneiting & Raftery, 2007; Parry et al., 2012), score matching (Hyvärinen, 2005), pseudolikelihood (Besag, 1975), loopy belief propagation (Murphy et al., 1999), and many, many more. Non-parametric methods (Gershman & Blei, 2012) can also be very effective¹.

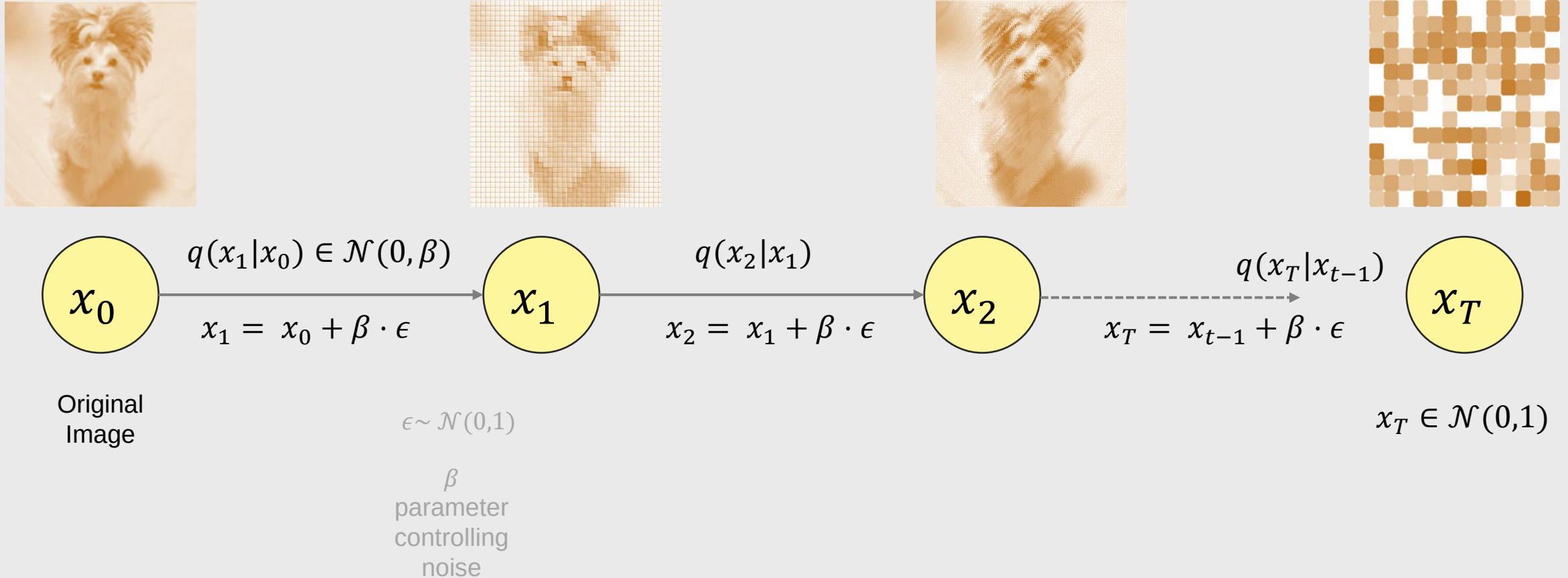
1.1. Diffusion probabilistic models

We present a novel way to define probabilistic models that allows:

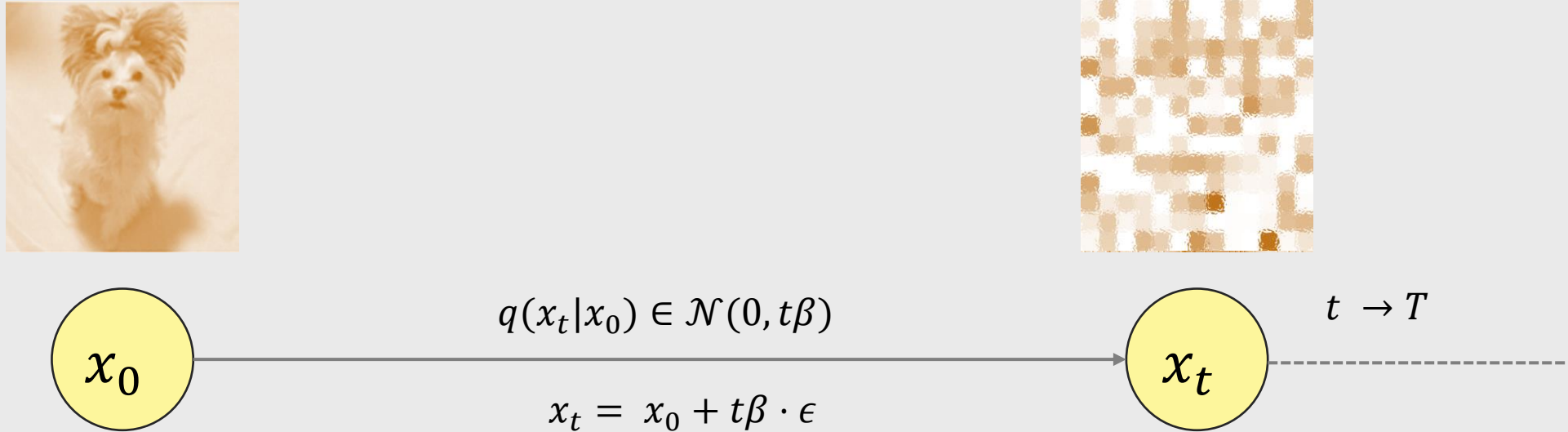
1. extreme flexibility in model structure,
2. exact sampling,

¹Non-parametric methods can be seen as transitioning smoothly between tractable and flexible models. For instance, a non-parametric Gaussian mixture model will represent a small amount of data using a single Gaussian, but may represent infinite data as a mixture of an infinite number of Gaussians.

Loss function in diffusion model



Loss function in diffusion model

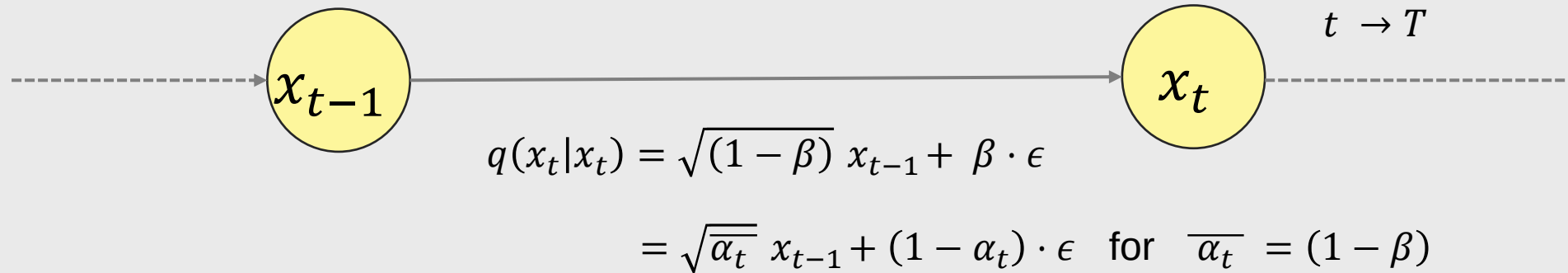
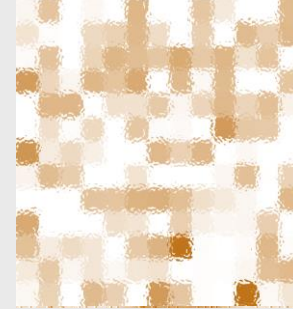


Original
Image

Objective
of forward diffusion

$$q(x_t|x_0) \xrightarrow{t \rightarrow \infty} \mathcal{N}(0,1)$$

Loss function in diffusion model



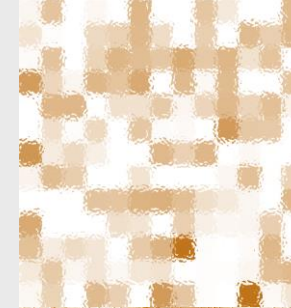
Loss function in diffusion model

3:33 PM

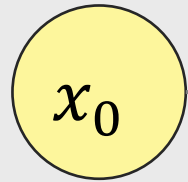


Objective

of forward diffusion



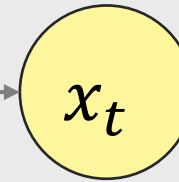
$$q(x_t|x_0) \xrightarrow{t \rightarrow \infty} \mathcal{N}(0,1)$$



Original
Image

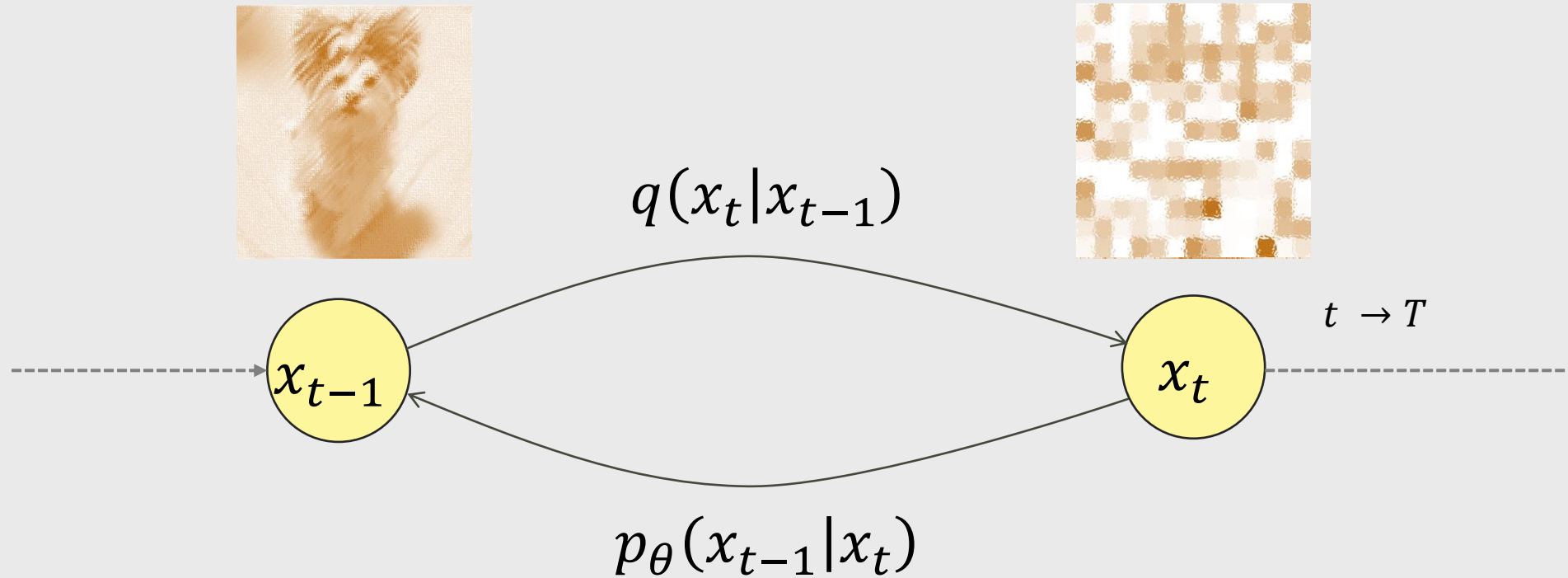
$$q(x_t|x_0) = \sqrt{(1-\beta)} x_0 + \beta \cdot \epsilon$$

$$= \sqrt{\bar{\alpha}_t} x_0 + (1 - \alpha_t) \cdot \epsilon \quad \text{for} \quad \bar{\alpha}_t = (1 - \beta_t)^t = \prod_{i=1}^t (1 - \beta_i)$$



$t \rightarrow T$

Loss function in diffusion model



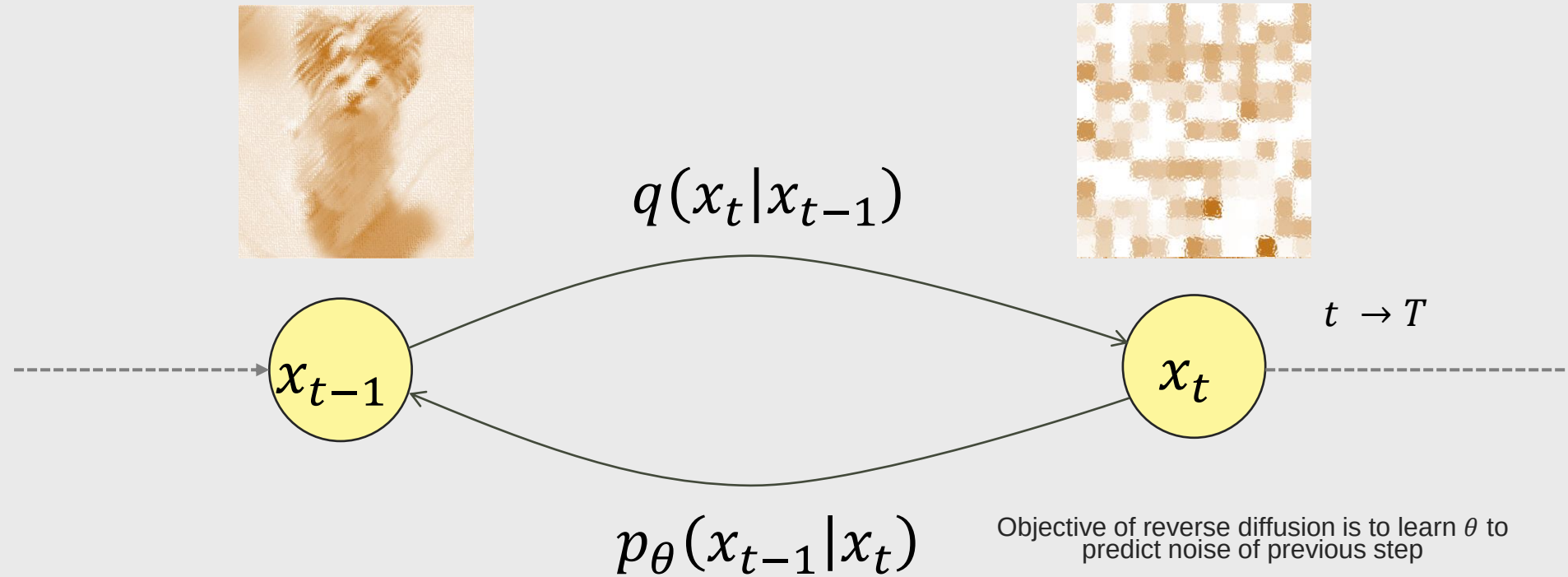
Objective

of reverse diffusion – learn θ to predict noise of previous step and

Minimise

$$-\log p_\theta(x_0)$$

Loss function in diffusion model



Complete Forward diffusion Processes

$$q(x_t \dots x_1 | x_0) = q(x_1 | x_0) q(x_2 | x_1) \dots q(x_T | x_{t-1}) = \prod_t^T q(x_t | x_{t-1})$$

Loss function in diffusion model

$$q(x_{1:T} | x_0) = \prod_t^T q(x_t | x_{t-1})$$

Complete forward diffusion processes

$$p_\theta(x_{0:T}) = p_\theta(x_T) \prod_t^T q(x_{t-1} | x_t)$$

Complete reverse diffusion processes

Objective of reverse diffusion – learn θ to predict noise of previous step to **minimise**

$$-\log p_\theta(x_0) = -\log \int p_\theta(x_{0:T}) dx_{1:T}$$

$$-\log p_\theta(x_0) = -\log \int q(x_{1:T} | x_0) \frac{p_\theta(x_{0:T})}{q(x_{1:T} | x_0)} dx_{1:T}$$

$$-\log p_\theta(x_0) \leq -\mathbb{E}_{q(x_{1:T} | x_0)} \left[\log \frac{p_\theta(x_{0:T})}{q(x_{1:T} | x_0)} \right]$$

Loss function in diffusion model

3:33 PM

$$L \leq -\mathbb{E}_{q(x_{1:T} | x_0)} \left[\log \frac{p_{\theta}(x_{0:T})}{q(x_{1:T} | x_0)} \right]$$

$$L = -\mathbb{E}_{q(x_{1:T} | x_0)} \left[\sum_{t>1} D_{\text{KL}}(q(x_{t-1} | x_t, x_0) || p_{\theta}(x_{0:T})) \right]$$

$$L = -\mathbb{E}_{q(x_{1:T} | x_0)} \left[\sum_{t>1} \frac{1}{2\sigma_t^2} \|\mu_t - \mu_{\theta}(x_t, t)\|^2 \right]$$

$$\mu_t = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{(1 - \bar{\alpha}_t)} \epsilon \right)$$

$$\mu_{\theta}(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{(1 - \bar{\alpha}_t)} \epsilon_{\theta}(x_t, t) \right)$$

$$L = -\mathbb{E}_{q(x_{1:T} | x_0)} \left[\sum_{t>1} \frac{\beta_t^2}{2\sigma_t^2 \alpha_t (1 - \bar{\alpha}_t)} \|\epsilon - \epsilon_{\theta}(x_t, t)\|^2 \right]$$

